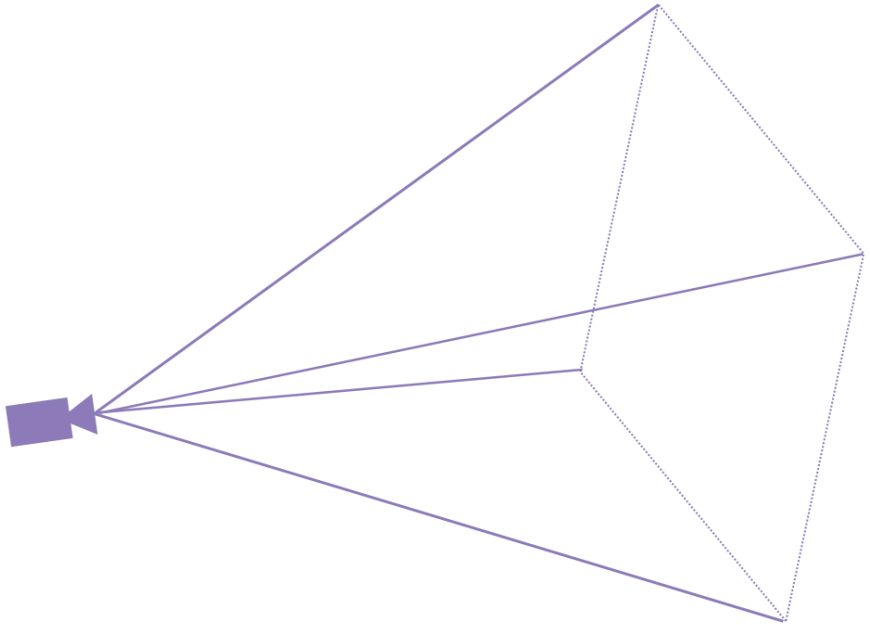
12. 원근 투영

화면에 현실감을 부여하는 변환

12.1 원근 투영 변환의 원리

- 투시 원근법
  - 3차원 공간 → 2차원 화면으로 표현하는 방법
  - 거리(깊이)에 따라 크기 변화 발생
  - 가까울수록 크게, 멀수록 작게 보임



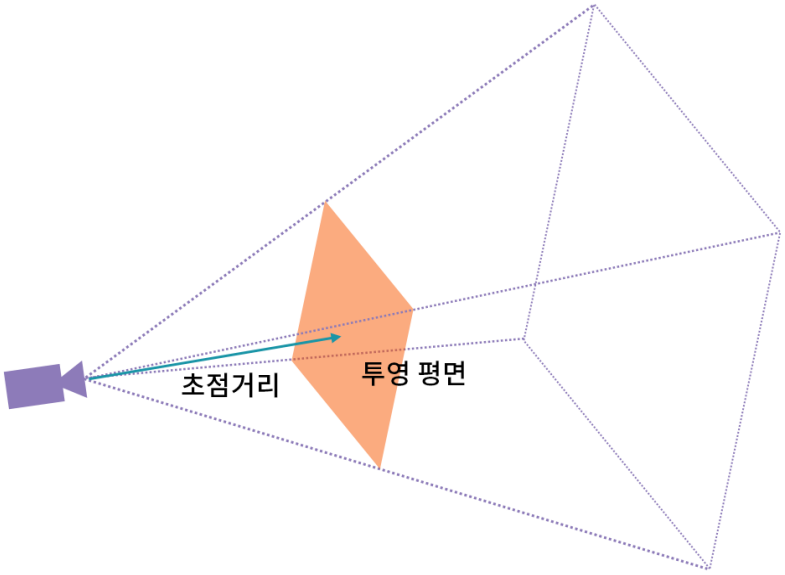
- 직교 투영 (Orthographic)
  - 크기 변화 없음
  - 평행선 유지

- 원근 투영 (Perspective)
  - 시점 기준 3차원 점을
    깊이(z)에 따라 축소하여 2차원 평면에 사상하는 변환

- 화각 (Field of View, FOV)
  - 카메라가 한 번에 볼 수 있는 시야의 각도
  - 값이 클수록 넓게 보이고, 작을수록 좁게 보임

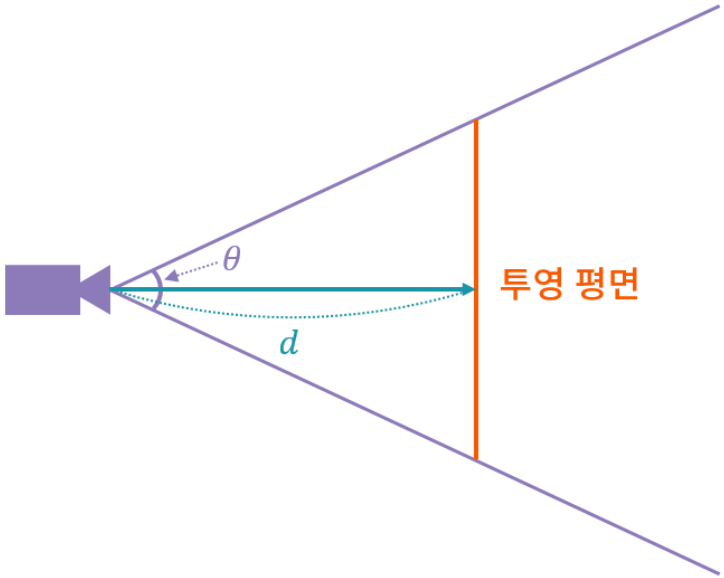
12.1 원근 투영 변환의 원리

- 투영 평면
  - 3D 공간의 점을 투영해서 그려지는 카메라 앞의 가상 화면
  - 보통 Near Plane



- 초점 거리 (Focal Length)
  - 카메라에서 투영 평면까지 거리

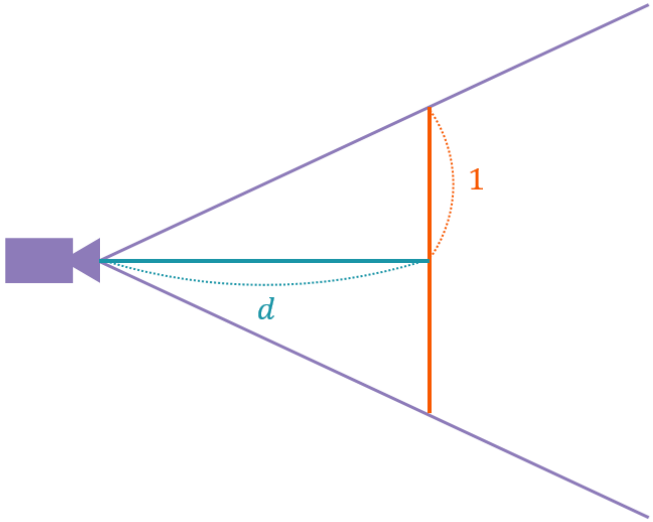
- 측면에서 바라본 투영 공간과 투영 평면

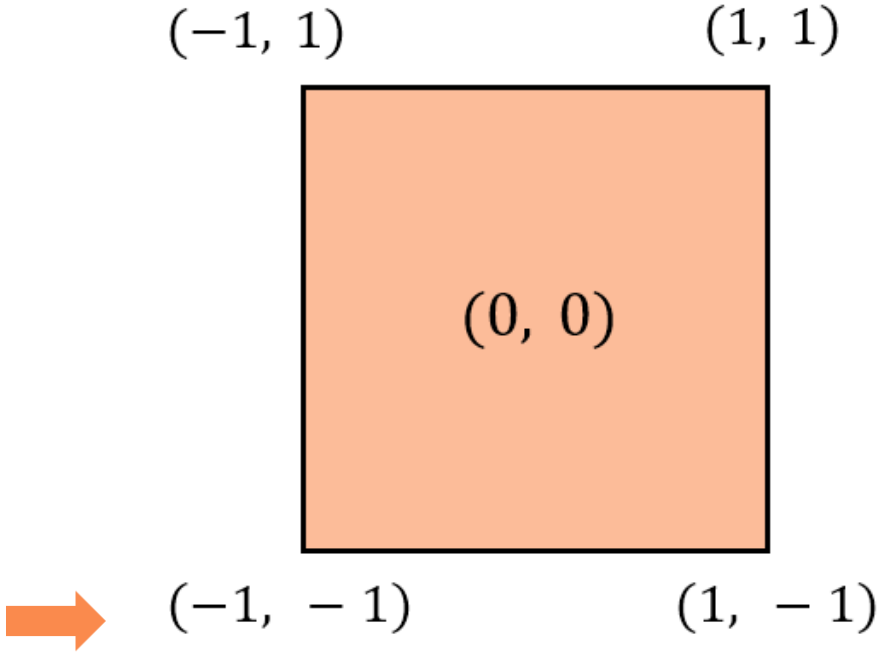


12.1 원근 투영 변환의 원리

- NDC 영역 (Normalized Device Coordinates)
  - 투영 변환 + Perspective Divide 이후의 좌표 공간
  - 화면 매핑 전의 정규화 좌표

- 좌표 범위
  - x∈[-1, +1], y∈[-1, +1]
  - z∈[0, +1] (DirectX)
  - z∈[-1, +1] (OpenGL)

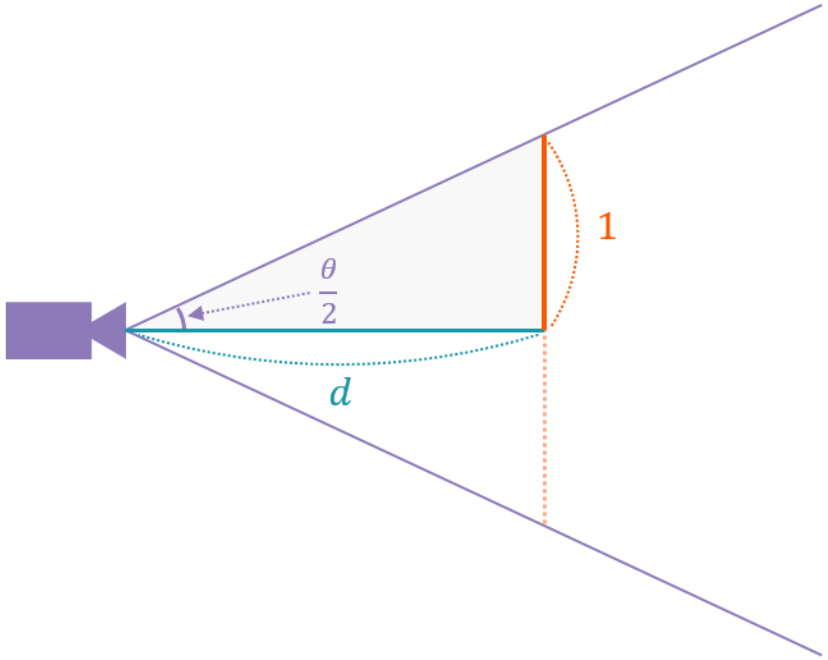




- 화각과 초점 거리의 관계

$$\tan\left(\frac{\theta}{2}\right) = \frac{1}{d}$$

$$d = \frac{1}{\tan\left(\frac{\theta}{2}\right)}$$



- 유클리드 공간 (Euclidean Space)
  - 거리와 각도가 정의되는 공간
  - 거리·각도 보존, 직선 최단 경로

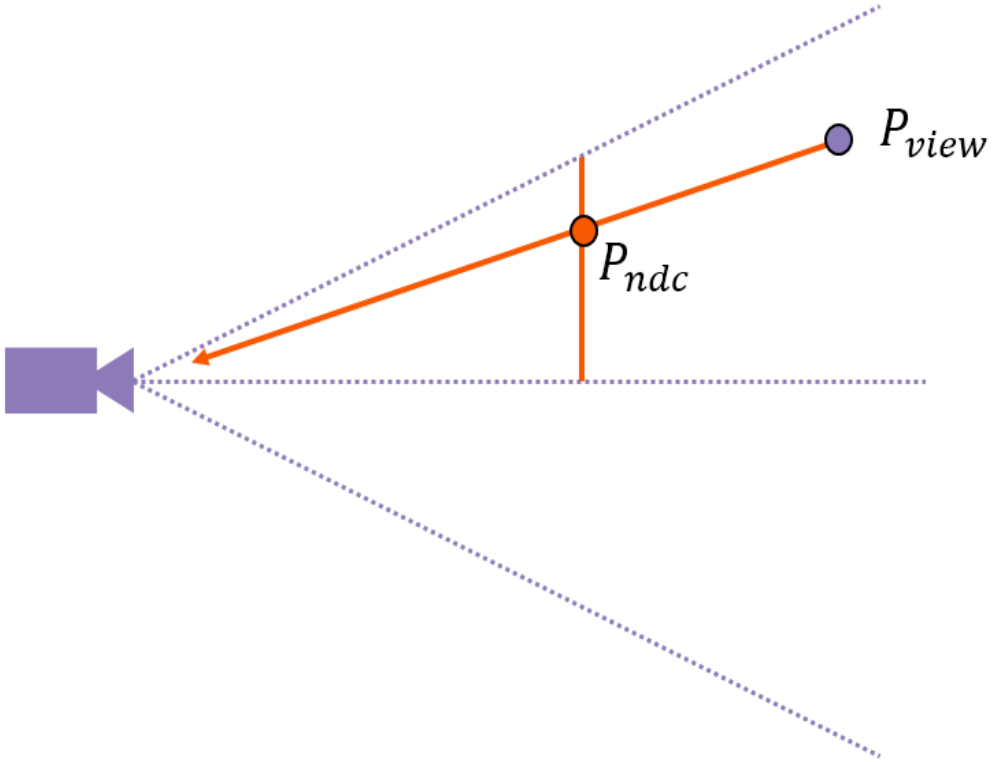
- 사영 공간 (Projective Space)
  - 원근 투영을 표현하는 공간
    → 원근 투영을 수학적으로 표현하는 공간

  - 평행선이 한 점에서 만난다고 가정
  - 길이·각도는 보존되지 않음, 비율만 유지

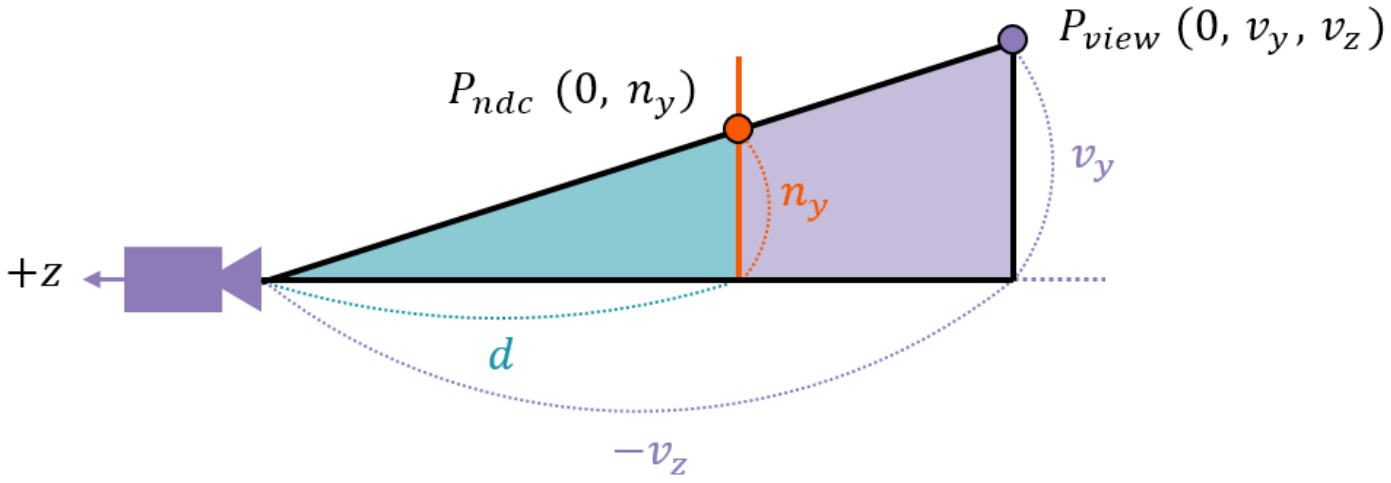
- 투영의 진행 과정

$$P_{view} = (0, v_y, v_z)$$

$$P_{ndc} = (0, n_y)$$



- 두 개의 닮은꼴 삼각형



$$n_y : d = v_y : -v_z$$

$$n_y = \frac{d \cdot v_y}{-v_z}$$

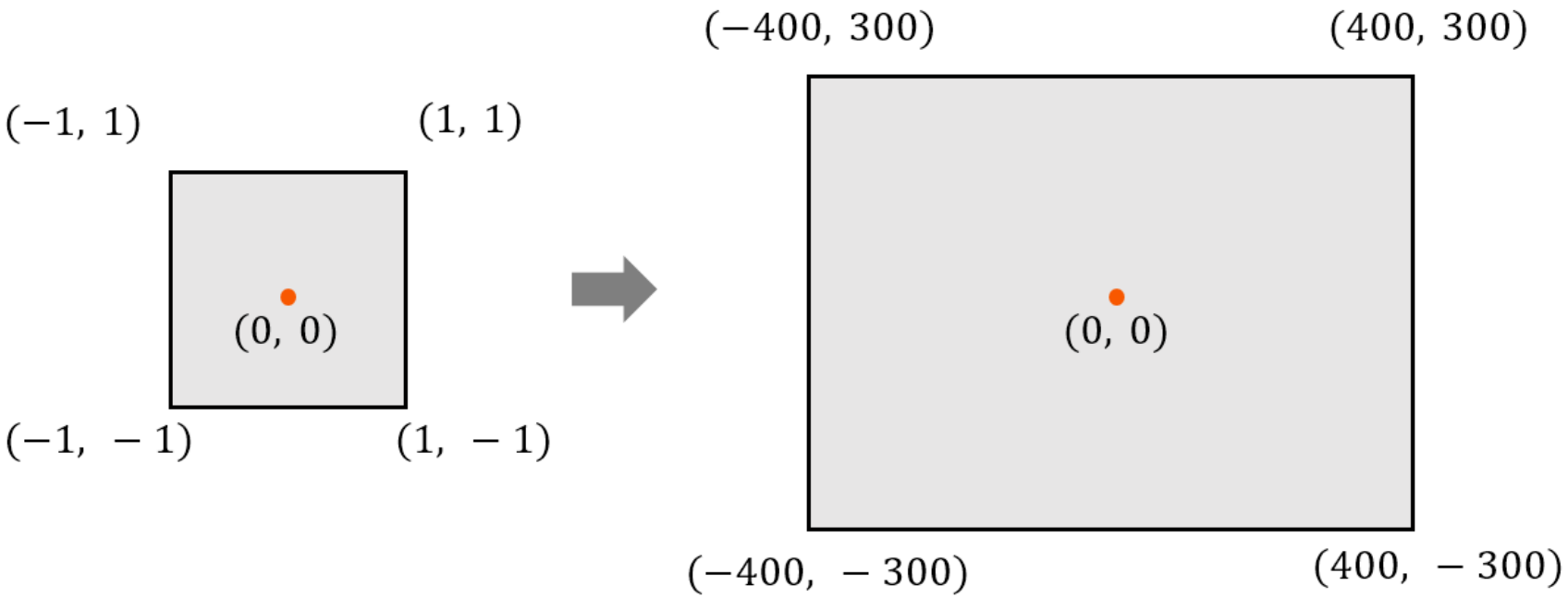
$$n_x = \frac{d \cdot v_x}{-v_z}$$

$$P_{ndc} = (n_x, n_y)$$

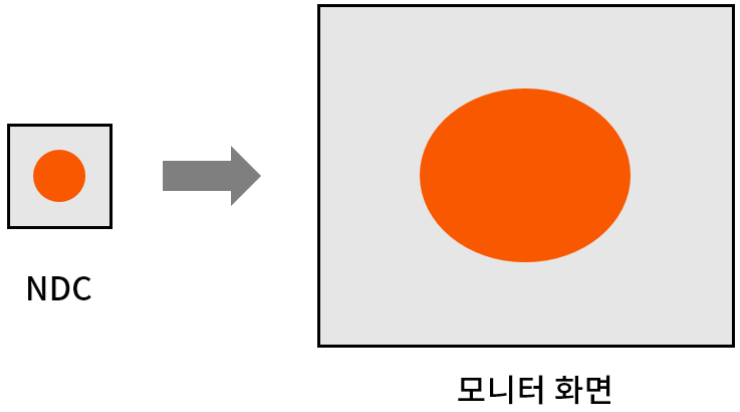
$$= \left(\frac{d \cdot v_x}{-v_z}, \frac{d \cdot v_y}{-v_z} \right)$$

$$= -\frac{d}{v_z}(v_x, v_y)$$

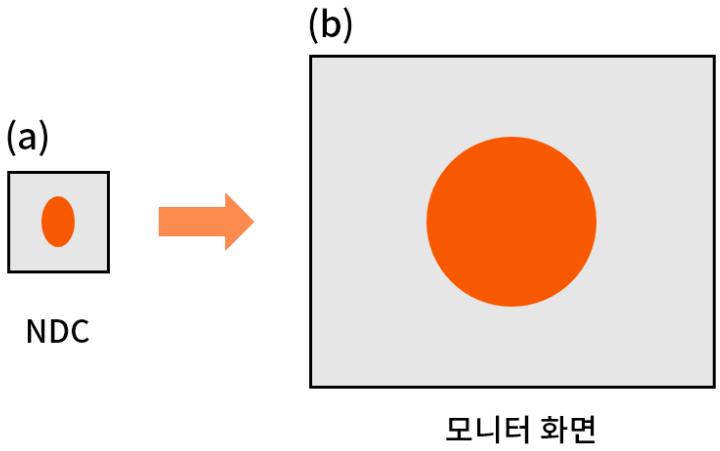
- NDC에서 화면 공간으로의 변환
  - ex) NDC → 800 X 600



- 종횡비를 고려하지 않고 그대로 확대하는 경우의 문제



- 종횡비(Aspect ratio) 적용



- 종횡비(Aspect ratio) 적용

$$P_{ndc} = -\frac{d}{v_z}(v_x, v_y) \rightarrow P_{ndc} = -\frac{d}{v_z}\left(\frac{v_x}{a}, v_y\right)$$

$$P_{ndc} = P \cdot \vec{v}$$

$$= \begin{bmatrix} \frac{1}{a} \cdot \frac{d}{-v_z} & 0 \\ 0 & \frac{d}{-v_z} \end{bmatrix} \cdot \begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

$$= \begin{bmatrix} \frac{d}{a} & 0 & 0 \\ 0 & d & 0 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} = \begin{bmatrix} \frac{d}{a} \cdot v_x \\ d \cdot v_y \\ -v_z \end{bmatrix}$$

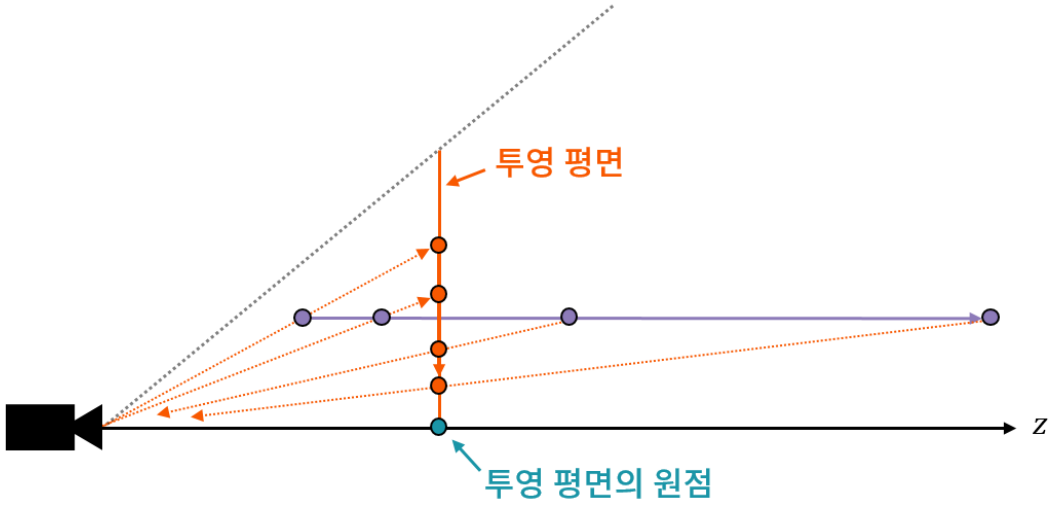
- 클립 좌표(Clip Coordinate)
  - 원근 투영 행렬 P로 변환되는 좌표계

$$P_{clip} = \left(\frac{d \cdot v_x}{a} , d \cdot v_y , -v_z \right)$$

- 클립 좌표(Clip Coordinate) → NDC 좌표
  - 마지막 $-v_z$ 로 나눔

$$P_{ndc} = \left(\frac{d \cdot v_x}{-a \cdot v_z} , \frac{d \cdot v_y}{-v_z} , 1 \right)$$

- 사영 공간 내에서 평행 이동하는 점에 따른 투영 결과



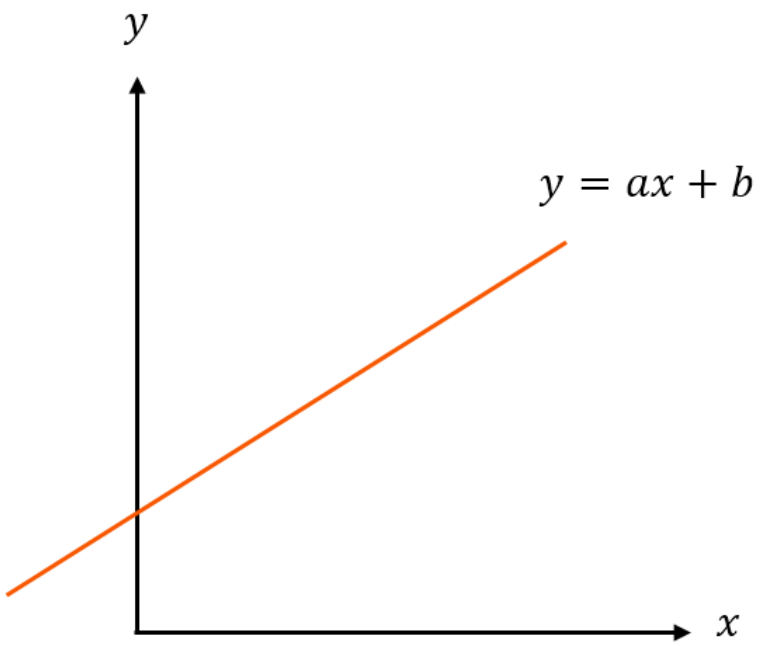
$$x = \frac{x'}{z'} \qquad y = \frac{y'}{z'}$$

- NDC 좌표에서의 직선 방정식

$$y = ax + b$$

$$\frac{y'}{z'} = a\frac{x'}{z'} + b$$

$$y' = ax' + bz'$$



- 동차 방정식
  - 모든 항이 같은 차수로 이루어진 방정식
    → 미지수에 대한 차수가 모두 동일
  - 스케일을 곱해도 형태가 변하지 않음

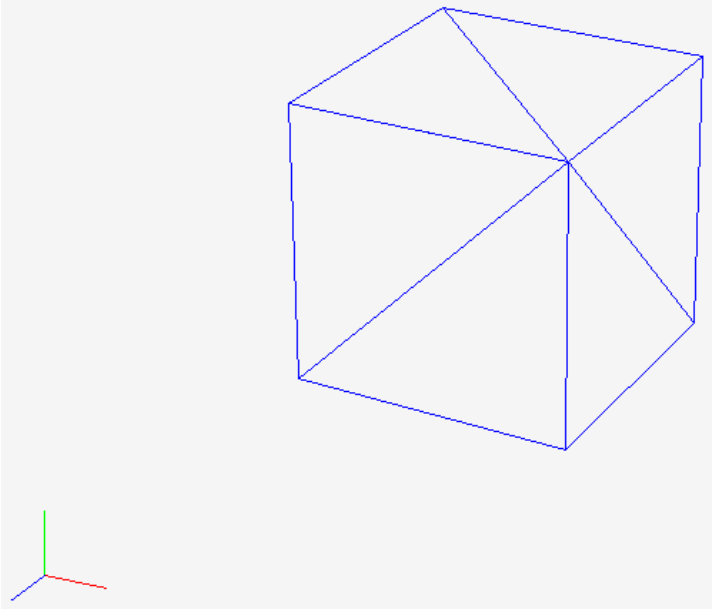
- 소실점 (Vanishing Point)
  - 평행한 직선들이 원근 투영에서 한 점으로 모여 보이는 점
  - 3차원 방향 정보가 2차원으로 투영되며 생김

- 미델하르니스의 가로수길

실습 12-1 화면에 원근감 부여하기

- 원근 투영 행렬

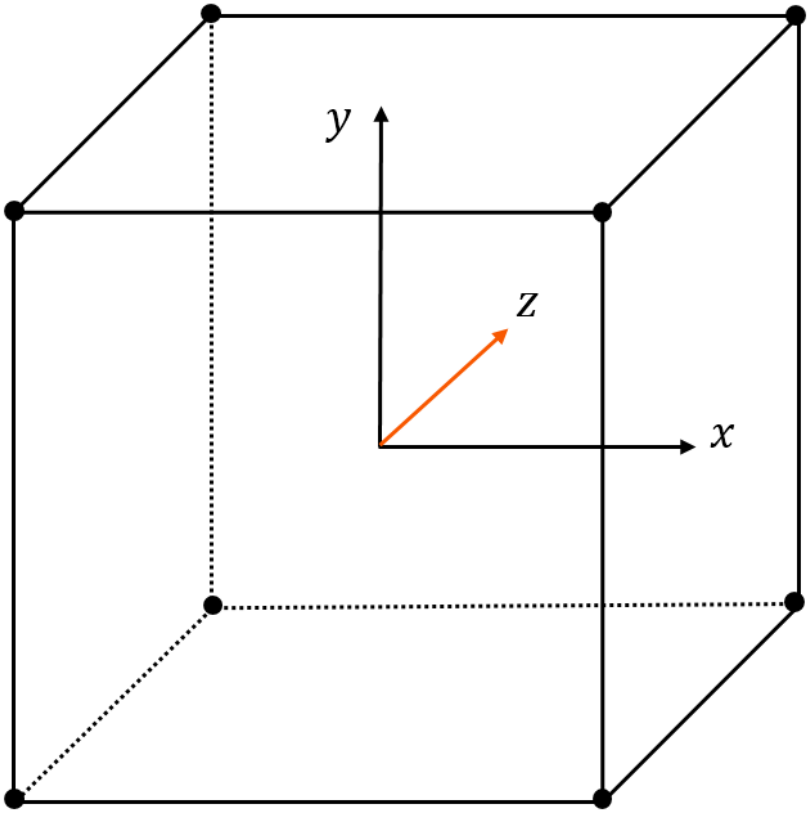
$$P = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



```
Matrix4x4 CameraObject::GetPerspectiveMatrix() const {

  float invA = 1.f / _ViewportSize.AspectRatio();
  float d = 1.f / tanf(Math::Deg2Rad(_FOV) * 0.5f);

  return Matrix4x4(
        Vector4::UnitX * invA * d,
        Vector4::UnitY        * d,
        Vector4(0.f, 0.f, -1.f, 0.f),
        Vector4(0.f, 0.f,    0, 1.f)
  );
}
```

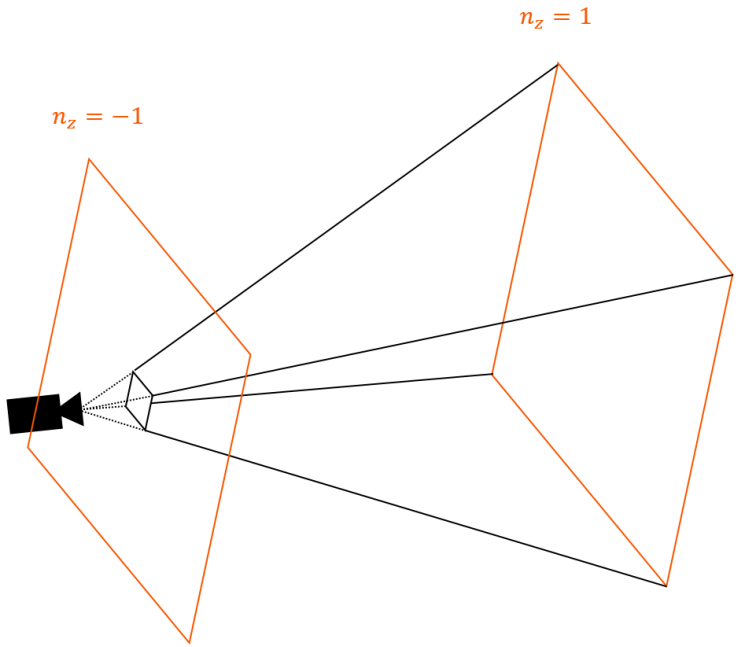
```
// 클립 좌표를 NDC 좌표로 변경
// 무한 원점인 경우, 약간 보정해준다.
if (v.Position.Z == 0.f)
    v.Position.Z = SMALL_NUMBER;

float invZ = 1.f / v.Position.Z;
v.Position.X *= invZ;
v.Position.Y *= invZ;
v.Position.Z *= invZ;
```

- 3차원으로 확장된 NDC 영역

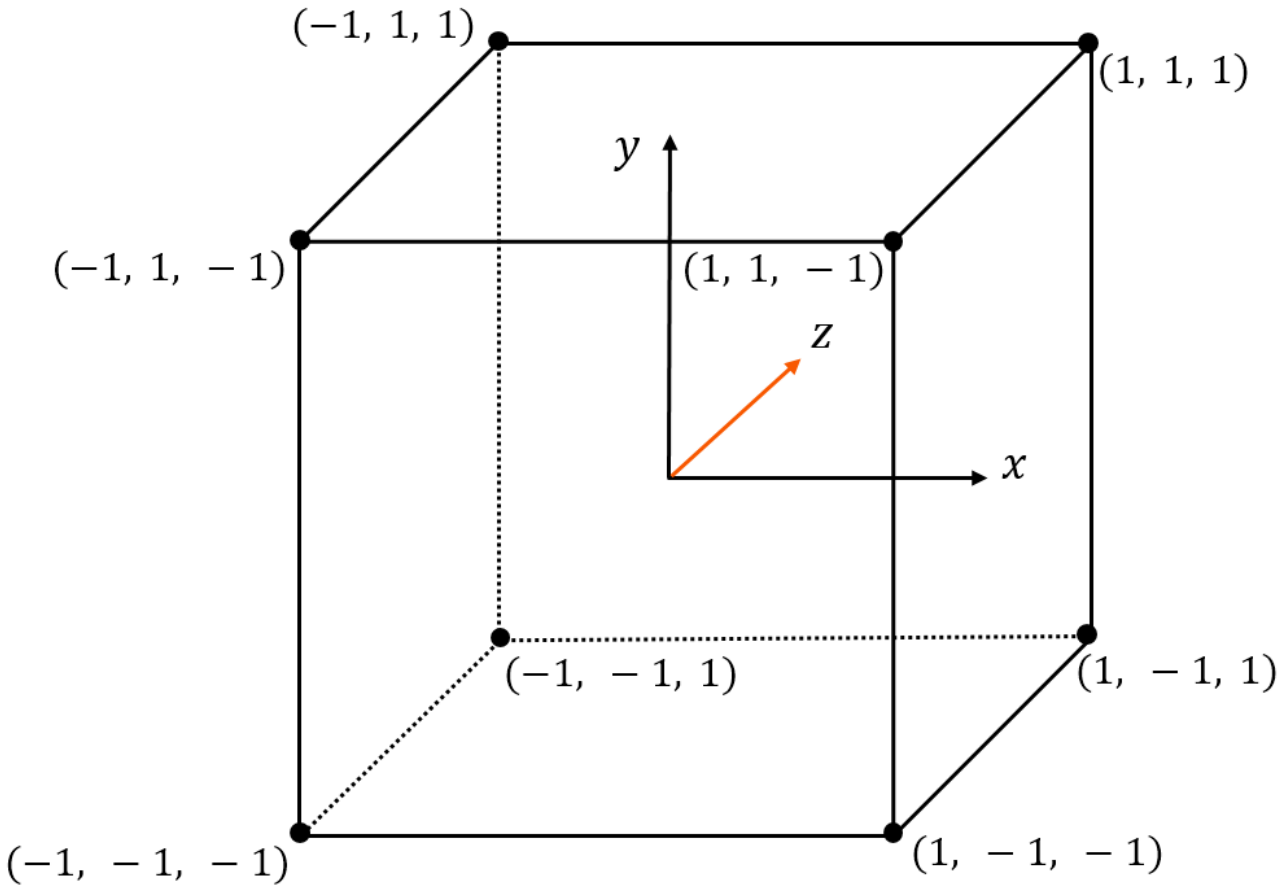


12.3 깊이 값

- 근 평면 (Near plane)
  - 카메라에서 가장 가까운 투영 기준 평면
  - 깊이 계산의 시작 기준
- 원 평면 (Far plane)
  - 카메라에서 가장 먼 투영 제한 평면
  - 깊이 계산의 끝 기준
- 절두체 (Frustum)
  - 원근 투영의 대상 공간
  - 근 평면과 원 평면 사이의 잘린 피라미드 형태의 공간
  - 카메라가 실제로 볼 수 있는 영역(View Volume)
  - 이 영역 안의 객체만 렌더링 → 밖에 있는 객체는 클리핑



- 3차원으로 확장한 NDC의 범위



- 3차원으로 확장한 NDC의 범위

$$P \cdot \vec{v} = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ i & j & k & l \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ v_z \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{d}{a} \cdot v_x \\ d \cdot v_y \\ ? \\ -v_z \end{bmatrix}$$

  - 깊이 값은 뷰 좌표계 의 x, y 축에 직교 ➔ i, j → 0
  - Near 평면 $(0,0,-n,1) \rightarrow (0,0,-1)$
  - Far 평면 $(0,0,f,1) \rightarrow (0,0,1)$

12.3 깊이 값

- Near 평면 $(0, 0, -n, 1) \rightarrow (0, 0, -1)$
- Far 평면 $(0, 0, f, 1) \rightarrow (0, 0, 1)$

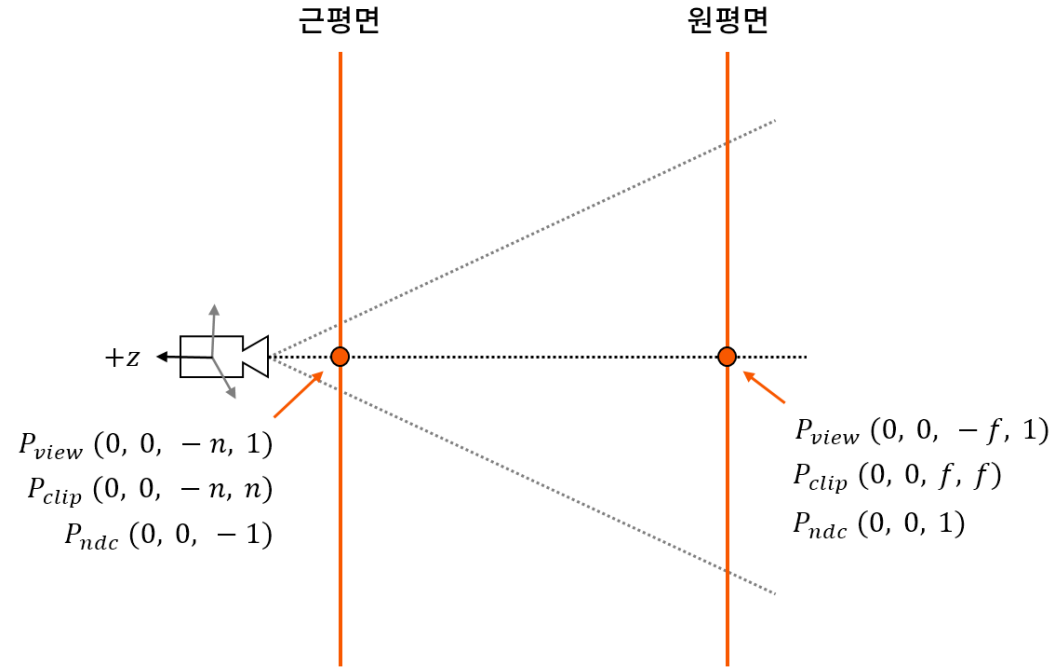
$$P_1 = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & k & l \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ -n \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ ? \\ n \end{bmatrix}$$

$$P_2 = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & k & l \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ -f \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ ? \\ f \end{bmatrix}$$

- 투영 행렬

$$P_1 = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & k & l \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ -n \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -n \\ n \end{bmatrix}$$

$$P_2 = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & k & l \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ -f \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ f \\ f \end{bmatrix}$$



- 투영 행렬 (Projection matrix)

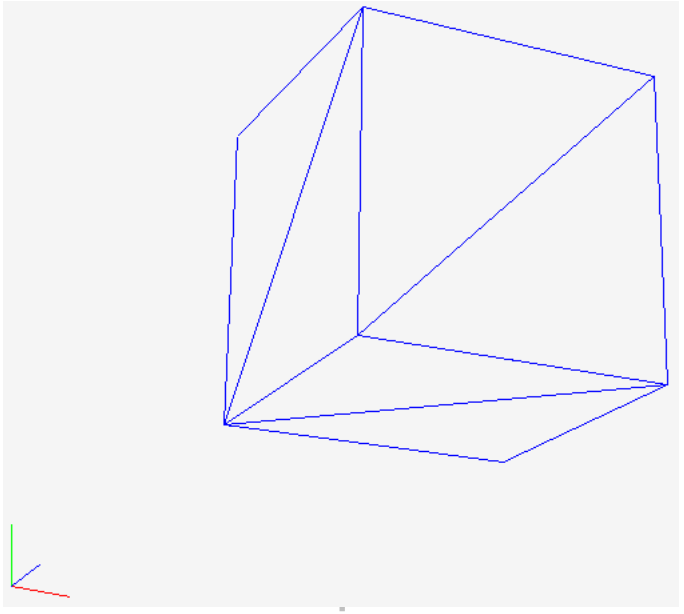
$$-kn + l = -n$$
$$-kf + l = f$$

$$k = \frac{-n - f}{-n + f} = \frac{n + f}{n - f}$$

$$l = f + kf = f + \frac{n + f}{n - f} f$$
$$= \frac{fn - ff + nf + ff}{n - f}$$
$$l = \frac{2nf}{n - f}$$

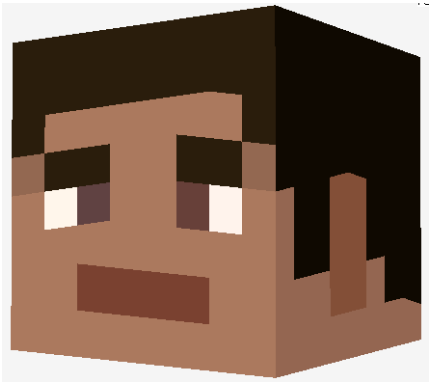
$$P = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & \frac{n+f}{n-f} & \frac{2nf}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

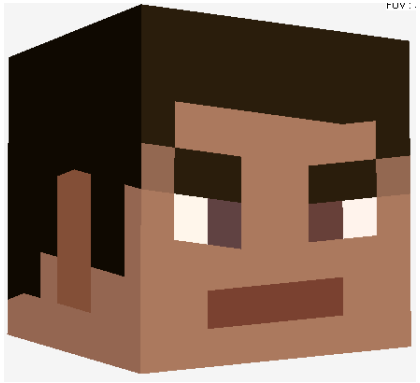
- 투영 행렬

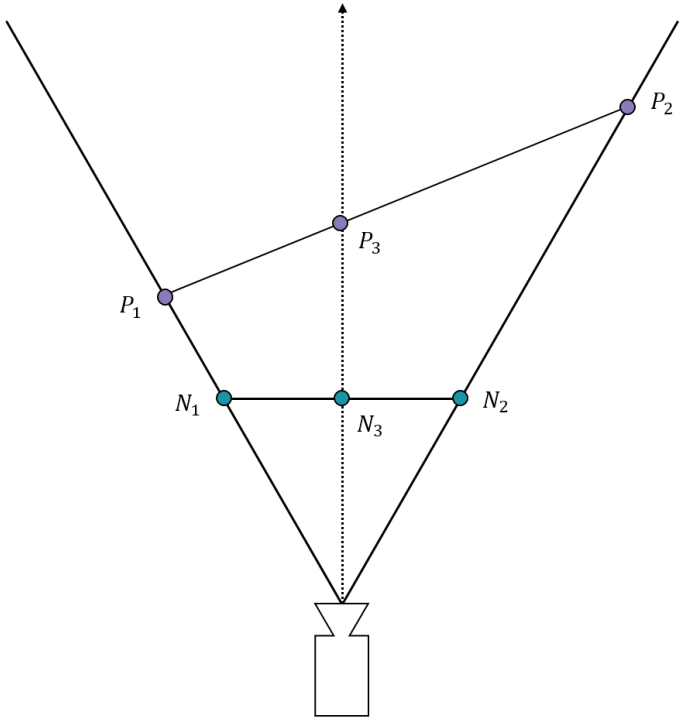


```
Matrix4x4 CameraObject::GetPerspectiveViewMatrix() const {
    // 뷰 행렬 관련 요소
    Vector3 viewX, viewY, viewZ;
    GetViewAxes(viewX, viewY, viewZ);
    Vector3 pos = _Transform.GetPosition();
    float zPos = viewZ.Dot(pos);

    // 투영 행렬 관련 요소
    float invA = 1.f / _ViewportSize.AspectRatio();
    float d = 1.f / tanf(Math::Deg2Rad(_FOV) * 0.5f);
    float dx = invA * d;
    float invNF = 1.f / (_NearZ - _FarZ);
    float k = (_FarZ + _NearZ) * invNF;
    float l = 2.f * _FarZ * _NearZ * invNF;

    return Matrix4x4(
    Vector4(dx * viewX.X, d * viewY.X, k * viewZ.X, -viewZ.X),
    Vector4(dx * viewX.Y, d * viewY.Y, k * viewZ.Y, -viewZ.Y),
    Vector4(dx * viewX.Z, d * viewY.Z, k * viewZ.Z, -viewZ.Z),
    Vector4(-dx * viewX.Dot(pos), -d * viewY.Dot(pos), -k * zPos + l, zPos)
    );
}
```

- 시야각에 걸쳐 있는 두 점과 이를 투영한 결과



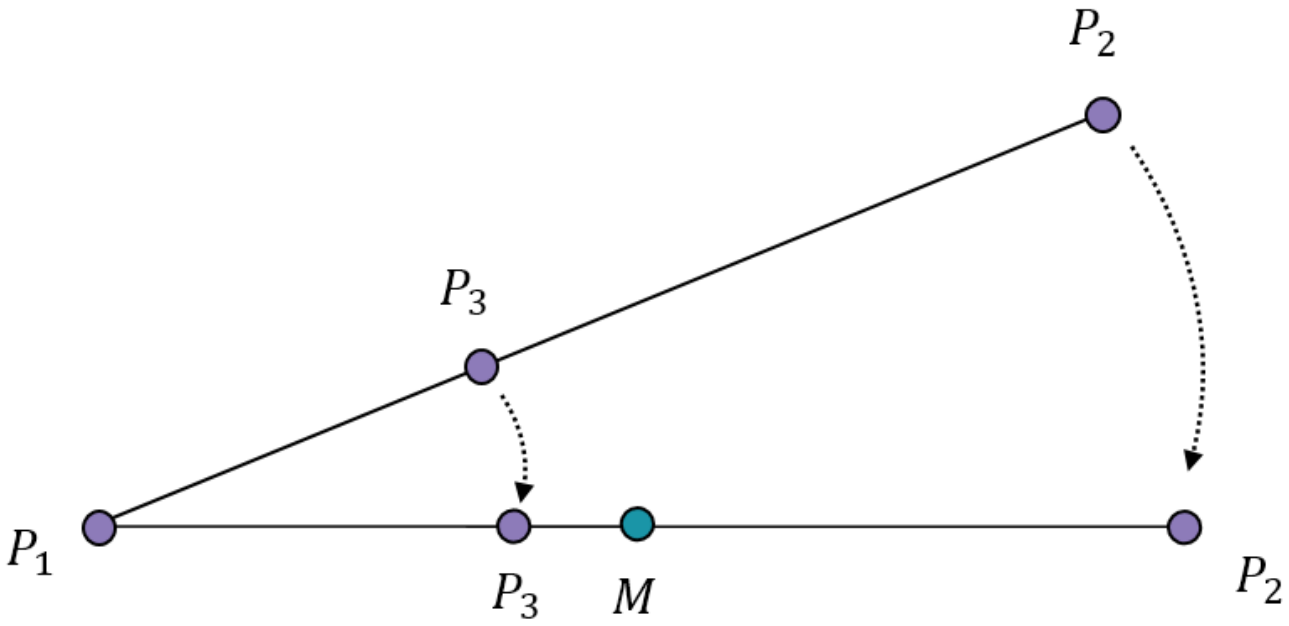




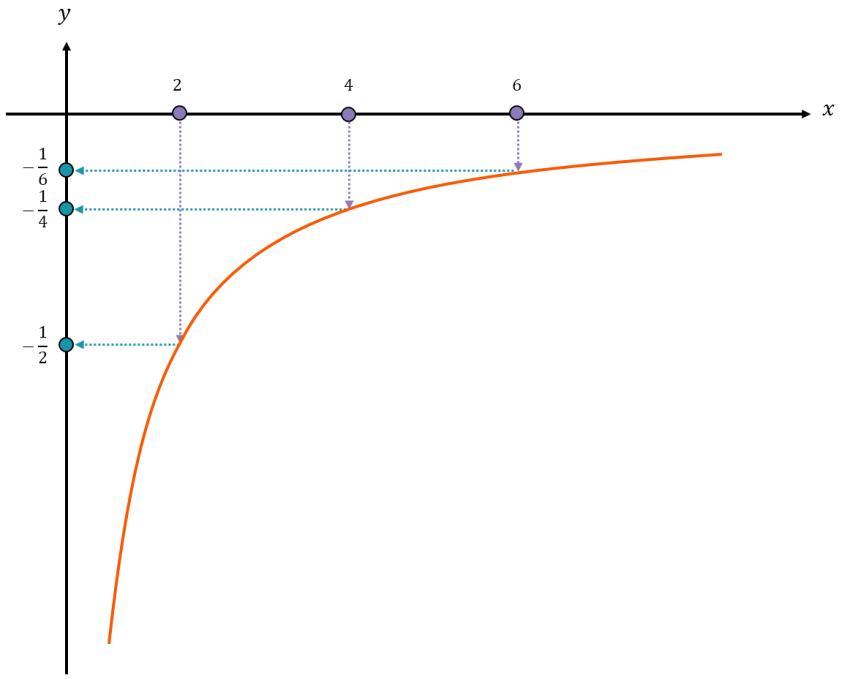
12.4 원근 보정 매핑

- 뷰 공간의 선을 수평으로 조정해 비교한 결과
  - NDC 의 무게 중심 좌표 ≠ 사영 공간 무게 중심 좌표

- 투영 보정 보간 (Perspective correction interpoliation)
  - NDC 무게 중심 좌표 → 사영 공간 무게 중심 좌표 계산
  - 투영 전 무게 중심 좌표 값 → 투영 보정 보간



- $y = -\frac{1}{x}$ 의 그래프



$$y' = q_1 \cdot y_1 + q_2 \cdot y_2$$

$$x' = t_1 \cdot x_1 + t_2 \cdot x_2$$

$$\frac{1}{x'} = q_1 \cdot \frac{1}{x_1} + q_2 \cdot \frac{1}{x_2}$$

- 투영 보정 보간 (Perspective correction interpolation)

$$y' = q_1 \cdot y_1 + q_2 \cdot y_2$$

$$x' = t_1 \cdot x_1 + t_2 \cdot x_2$$

$$\frac{1}{x'} = q_1 \cdot \frac{1}{x_1} + q_2 \cdot \frac{1}{x_2}$$

$$x' = \frac{1}{q_1 \cdot \frac{1}{x_1} + q_2 \cdot \frac{1}{x_2}}$$

$$x' \left(q_1 \cdot \frac{1}{x_1} + q_2 \cdot \frac{1}{x_2} \right) = 1$$

$$q_1 \cdot \frac{x'}{x_1} + q_2 \cdot \frac{x'}{x_2} = 1$$

$$q_1 \cdot \frac{x'}{x_1} + q_2 \cdot \frac{x'}{x_2} = t_1 + t_2$$

$$t_1 = \frac{x'}{x_1} q_1 \qquad t_2 = \frac{x'}{x_2} q_2$$

- 3점으로 확대

$$x' = \frac{1}{q_1 \cdot \frac{1}{x_1} + q_2 \cdot \frac{1}{x_2} + q_3 \cdot \frac{1}{x_3}}$$

$$t_1 = \frac{x'}{x_1} q_1 \quad t_2 = \frac{x'}{x_2} q_2 \quad t_3 = \frac{x'}{x_3} q_3$$

- x → −z 로 치환, 원근 보정 매핑 완성

$$z' = \frac{1}{q_1 \cdot \frac{1}{z_1} + q_2 \cdot \frac{1}{z_2} + q_3 \cdot \frac{1}{z_3}}$$

$$t_1 = \frac{z'}{z_1} q_1 \quad t_2 = \frac{z'}{z_2} q_2 \quad t_3 = \frac{z'}{z_3} q_3$$

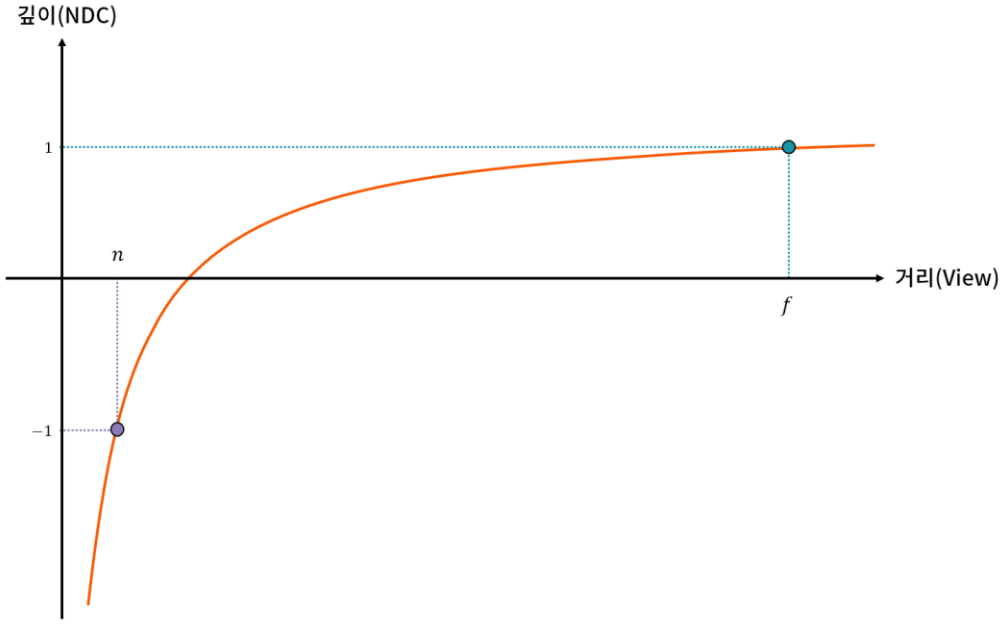
- 원근 보정 매핑

```
float s = (wdotv * udotv - wdotu * vdotv) * invDenominator;
float t = (wdotu * udotv - wdotv * udotu) * invDenominator;
float oneMinusST = 1.f - s - t;

if (((s >= 0.f) && (s <= 1.f)) && ((t >= 0.f) && (t <= 1.f))
                && ((oneMinusST >= 0.f) && (oneMinusST <= 1.f)))
{
  // 투영보정에 사용할 공통 분모
  float z = invZ0 * oneMinusST + invZ1 * s + invZ2 * t;
  float invZ = 1.f / z;

  Vector2 targetUV = (InVertices[0].UV * oneMinusST * invZ0
                  + InVertices[1].UV * s * invZ1 + InVertices[2].UV * t * invZ2) * invZ;

r.DrawPoint(fragment, FragmentShader3D(mainTexture.GetSample(targetUV), LinearColor::White));
```

12.5 깊이 버퍼

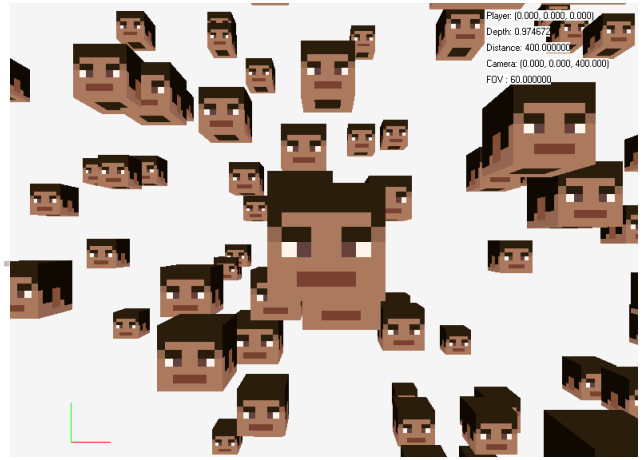
- 깊이 버퍼 (Depth Buffer, Z-Buffer)
  - 픽셀마다 깊이(z) 값을 저장하는 버퍼
  - 더 가까운 픽셀만 화면에 표시, 어떤 물체가 앞에 보일지 결정
    - 기존 깊이 vs 새 깊이 비교
    - 더 가까우면 갱신, 아니면 버림
  - 절두체(Near~Far) 범위 깊이 사용

- 뷰 공간 좌표값 NDC 좌표값 비교



- 삼각형 3 점 z_1, z_2, z_3 의 깊이 값

$$z' = q_1 \cdot z_1 + q_2 \cdot z_2 + q_3 \cdot z_3$$




```
// 투영보정에 사용할 공통 분모
float z = invZ0 * oneMinusST + invZ1 * s + invZ2 * t;
float invZ = 1.f / z;

// 깊이 버퍼 테스팅
if (toggleDepthTesting)
{
  // 깊이 테스팅

float newDepth = InVertices[0].Position.Z * oneMinusST + InVertices[1].Position.Z
 * s
                    + InVertices[2].Position.Z * t;
  float prevDepth = r.GetDepthBufferValue(fragment);

  if (newDepth < prevDepth)
  {
    // 픽셀을 처리하기 전 깊이 값을 버퍼에 보관
    r.SetDepthBufferValue(fragment, newDepth);
  }
  else
  {
    // 이미 앞에 무언가 그려져있으므로 픽셀그리기는 생략
    continue;
```

- 깊이 색상
  - n_z 로 하면 구분 안감

$$C_{depth} = (n_z + 1) \cdot 0.5$$

  - 뷰공간 깊이 값 사용

$$z' = \frac{1}{q_1 \cdot \frac{1}{z_1} + q_2 \cdot \frac{1}{z_2} + q_3 \cdot \frac{1}{z_3}} \qquad C_{depth} = \frac{z' - n}{f - n}$$

```
if (IsDepthBufferDrawing())
{
  float grayScale = (newDepth + 1.f) * 0.5f;
  if (useLinearVisualization)
  {
    // 카메라로부터의 거리에 따라 균일하게 증감하는 흑백 값으로 변환
    grayScale = (invZ - n) / (f - n);
  }

  // 뎁스 버퍼 그리기
  r.DrawPoint(fragment, LinearColor::White * grayScale);
```

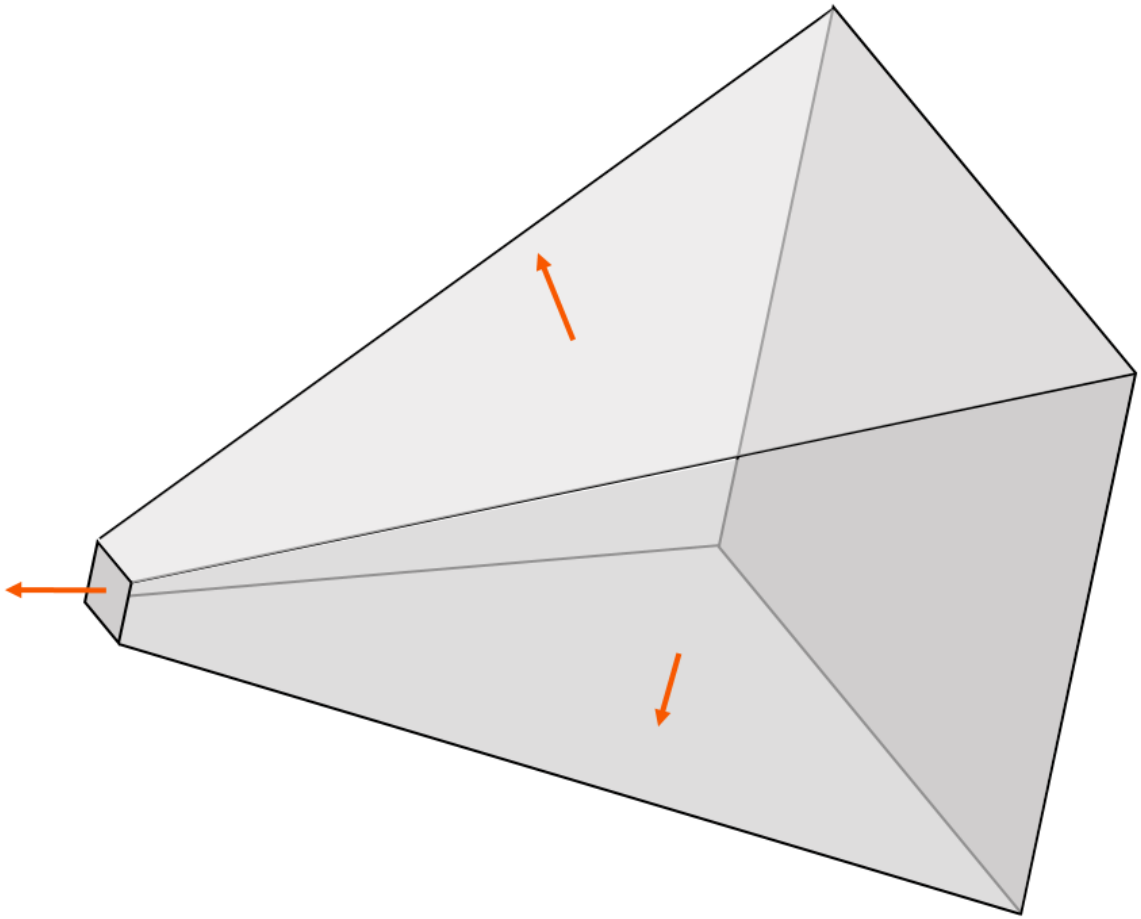
12.6 정리

- 원근 투영 행렬 (Perspective Projection Matrix)
  - 3D 좌표를 원근이 적용된 Clip Space로 변환
  - w로 나누어 원근 효과 생성
- 원근 보정 매핑 (Perspective-Correct Mapping)
  - 텍스처 좌표를 원근 왜곡 없이 보간
  - 1/w를 이용해 선형 보간 수행
- 깊이 버퍼 (Depth Buffer)
  - 픽셀마다 깊이 값을 저장하여 가시성 결정
  - 더 가까운 픽셀만 화면에 표시

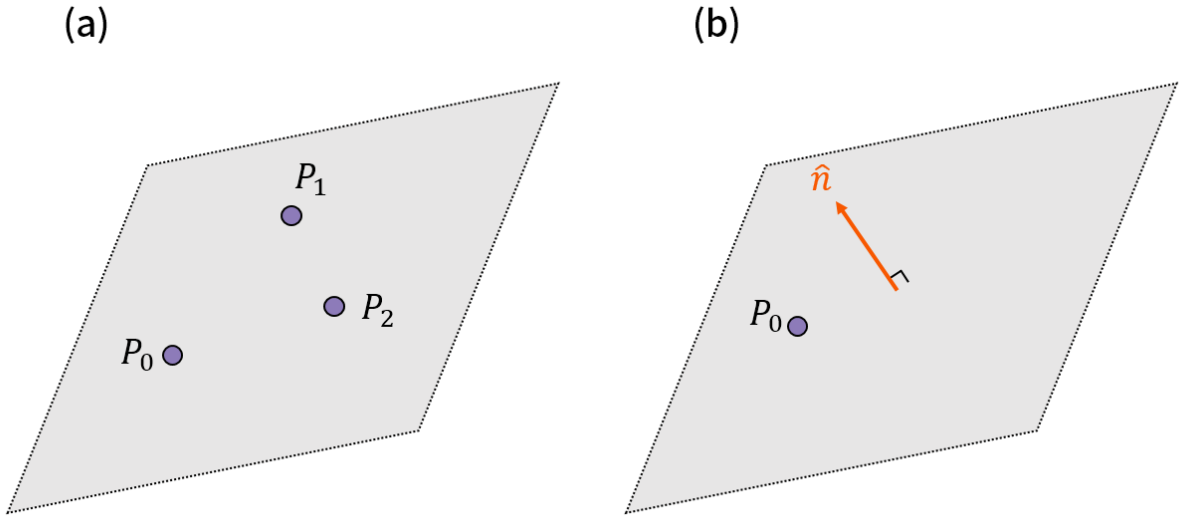
13. 절두체

최적화된 3차원 공간

- 절두체를 구성하는 6개의 평면



- 평면을 이루는 최소 구성 요소



$$\hat{n} \cdot (P - P_0)$$

$$= (a,\ b,\ c) \cdot (x - x_0,\ y - y_0,\ z - z_0) = 0$$

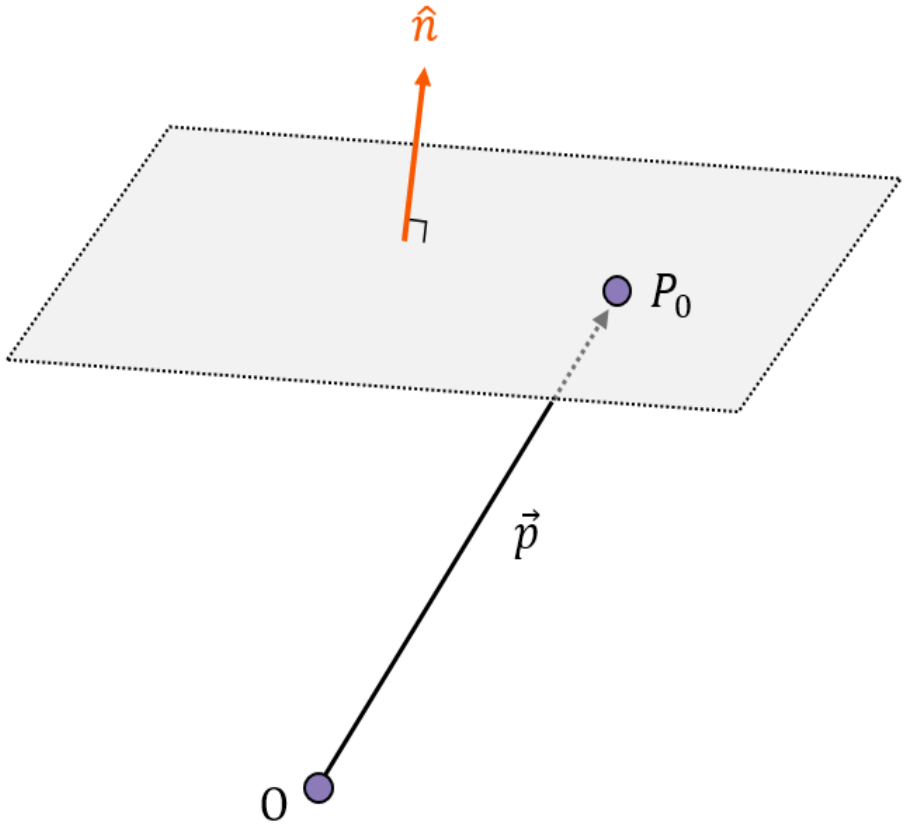
$$= ax + by + cz - (ax_0 + by_0 + cz_0) = 0$$

$$ax + by + cz + \mathrm{d} = 0$$

- D를 계산하기 위한 평면의 구성 요소

$$ax + by + cz + \mathrm{d} = 0$$

$$d = \hat{n} \cdot \overrightarrow{0P_0} = \hat{n} \cdot \vec{p}$$

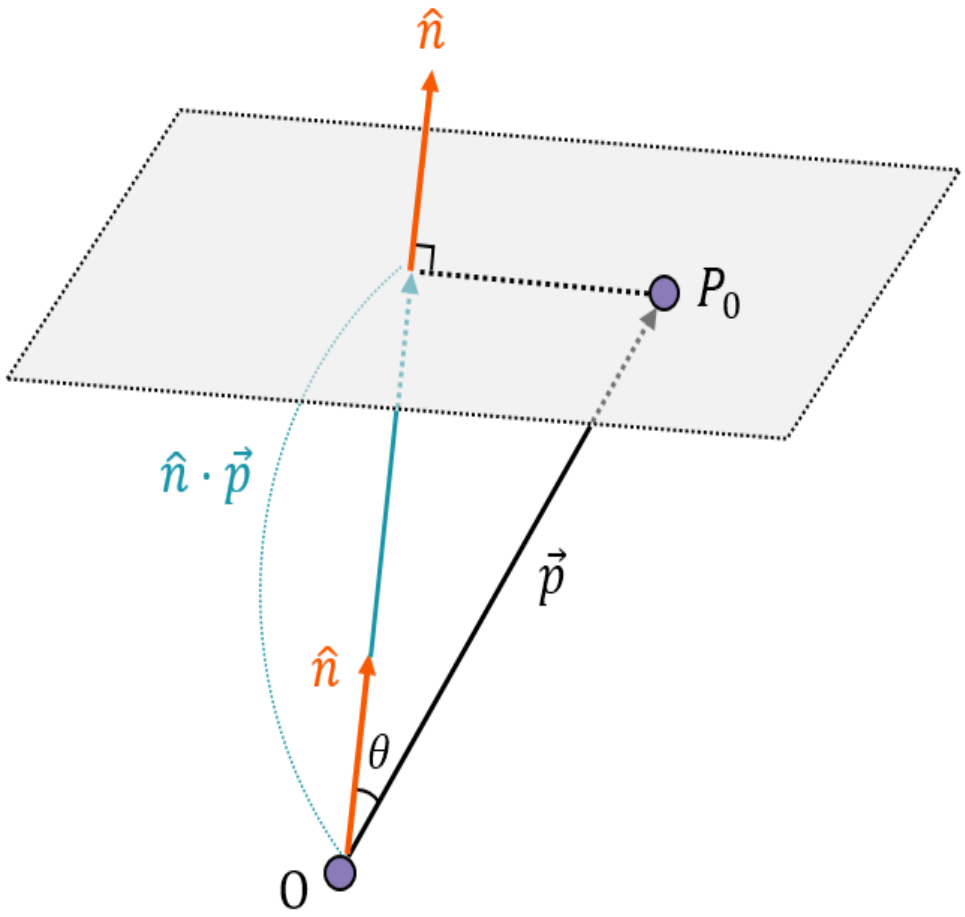


13.1.1 평면의 방정식

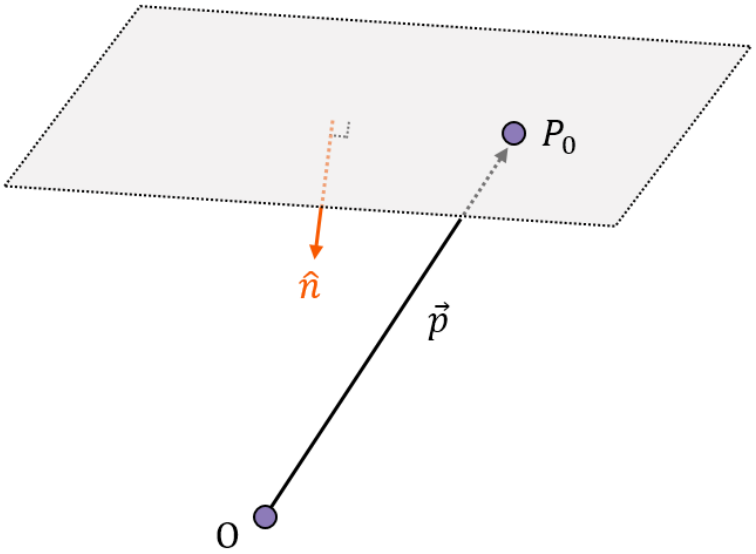
- 상수 d와 관련있는 두 벡터의 내적

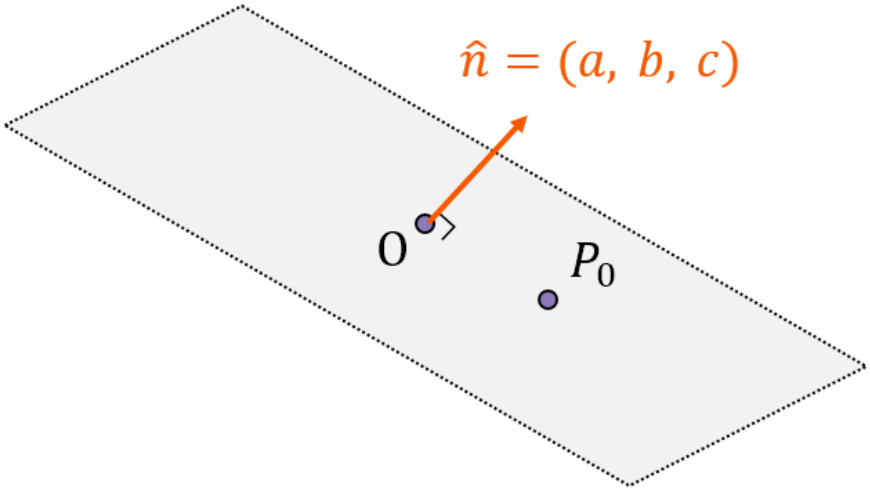
- d 의 의미
  → 원점에서의 최단 거리

$$\hat{n} \cdot \vec{p} = |\vec{p}| \cos\theta$$



- d 의 부호
  - $0 < d$: 평면의 방향(법선)이 원점을 향함
    → 원점이 평면 위에
  - $0 > d$: 평면의 방향(법선)이 원점에서 멀어지는 방향
    → 원점이 평면 아래
  - $0 == d$: → 원점이 평면에



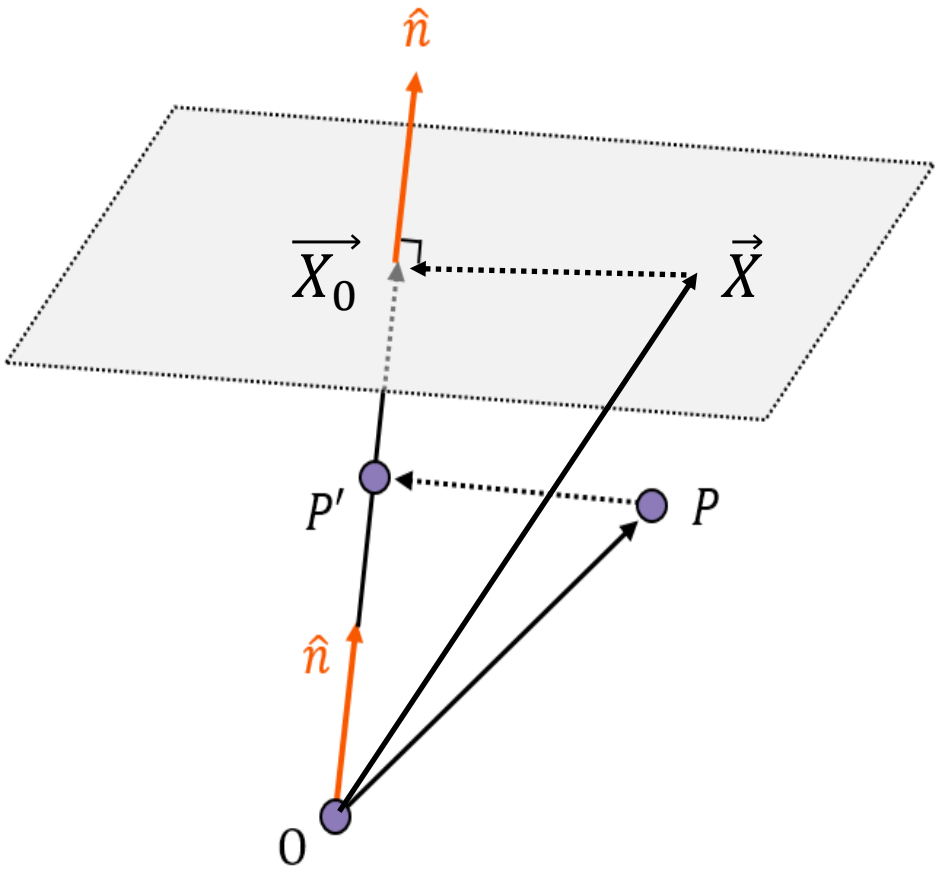


- **점의 평면 아래**

$$\hat{n} \cdot \vec{X} = \hat{n} \cdot \overrightarrow{X_0}$$
$$\hat{n} \cdot \overrightarrow{0\mathrm{P}} = \hat{n} \cdot \overrightarrow{0\mathrm{P}'}$$
$$\hat{n} \cdot \overrightarrow{0P'} < \hat{n} \cdot \overrightarrow{X_0}$$
$$\hat{n} \cdot \overrightarrow{0P'} - \hat{n} \cdot \overrightarrow{X_0} < 0$$

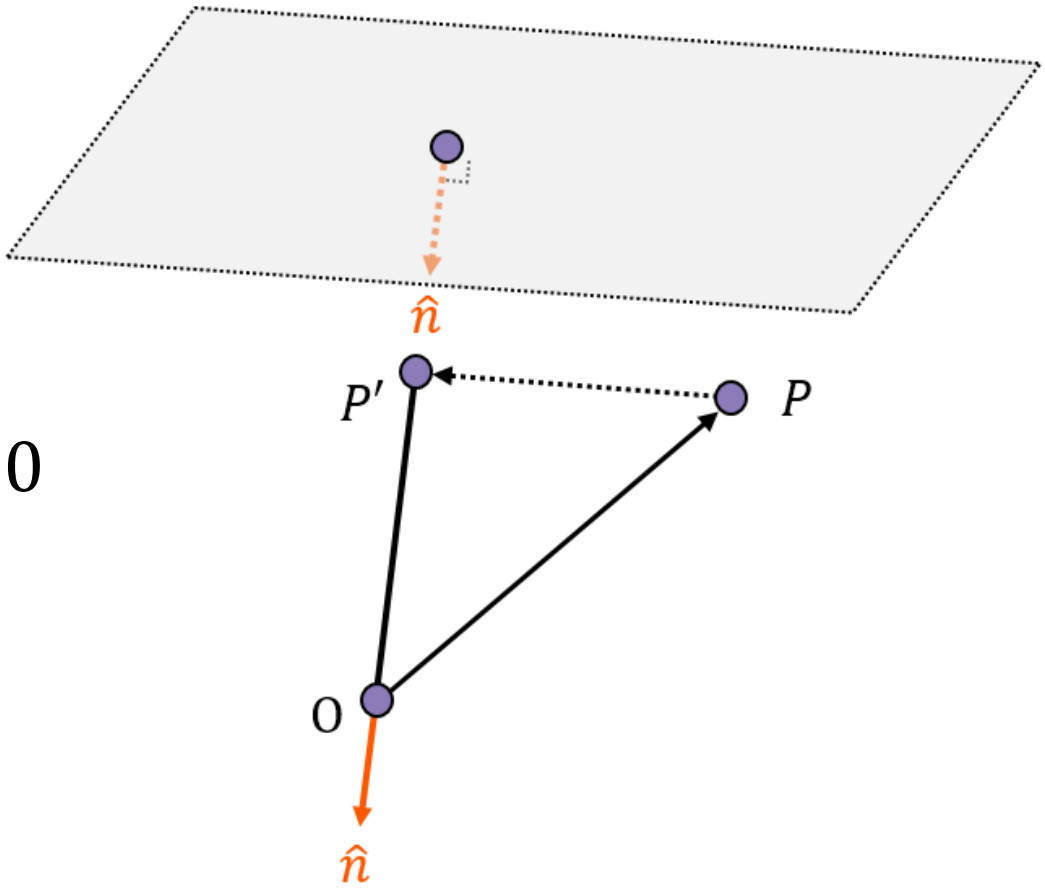
- **점의 평면 위**

$$\hat{n} \cdot \overrightarrow{0P'} - \hat{n} \cdot \overrightarrow{X_0} > 0$$



- 법선 벡터가 반대 방향인 경우
  → 동일하게 적용

$$\hat{n} \cdot \overrightarrow{0P'} + d > 0$$



13.1.1 평면의 방정식

- 점의 평면 위
  - $(a,\ b,\ c) \cdot (x_0,\ y_0,\ z_0) + \ d > 0$
- 점의 평면 아래
  - $(a,\ b,\ c) \cdot (x_0,\ y_0,\ z_0) + \ d < 0$
- 점이 평면 안에
  - $(a,\ b,\ c) \cdot (x_0,\ y_0,\ z_0) + \ d < 0$
- 정규화된 법선 벡터를 가진 평면 에서 점까지 최단 거리

$$distance = |ax_0 + by_0 + cz_0 + d|$$

- 평면의 법선 벡터의 정규화까지 고려한다면

$$distance = \frac{|ax_0 + by_0 + cz_0 + d|}{\sqrt{a^2 + b^2 + c^2}}$$

- **평면의 방정식**
  - $ax + by + cz + \mathrm{d} = 0$

$$\vec{N} = (a, b, c)$$

$$\vec{N} \cdot (x, y, z) + d = 0$$

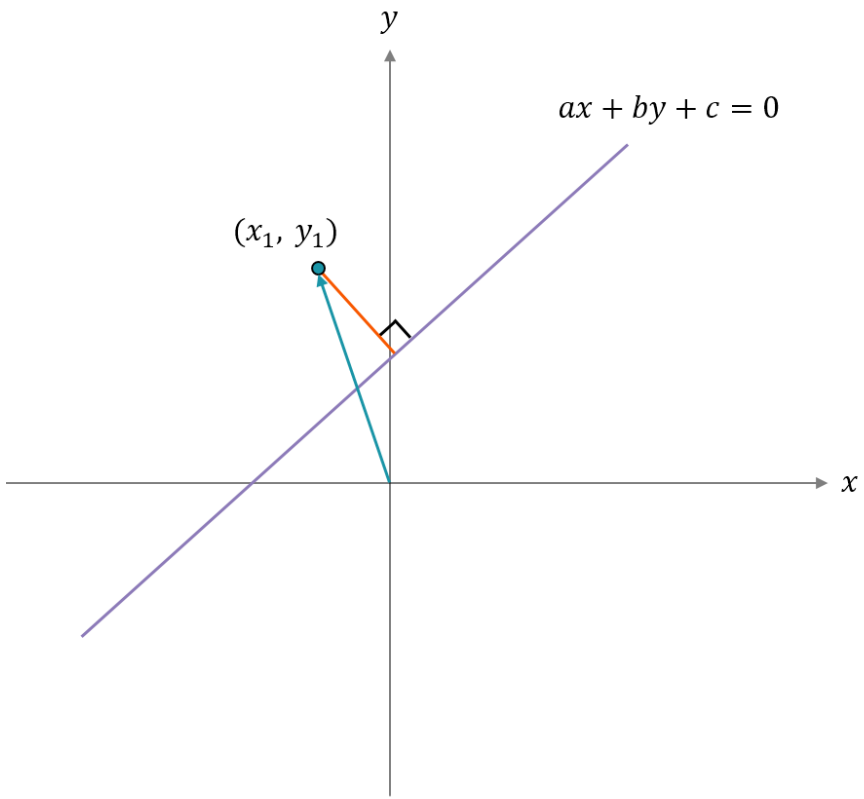
$$\vec{N} \cdot \vec{\mathrm{X}} + d = 0$$

$$\frac{\vec{N} \cdot \vec{X} + d}{|\vec{N}|} = 0 \quad \Rightarrow \quad \frac{ax + by + cz + d}{\sqrt{a^2 + b^2 + c^2}} = 0$$

- 2차원공간에서 직선과 점 사이의 최단 거리
  → 3차원 평면에서 z → 0 인 평면과의 거리

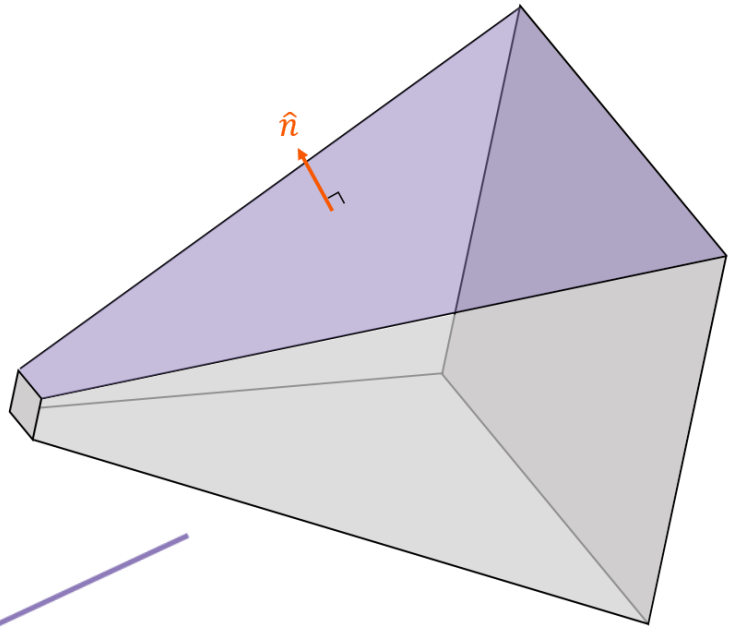
$$distance = \frac{|ax_0 + by_0 + cz_0 + d|}{\sqrt{a^2 + b^2 + c^2}}$$

$$distance = \frac{|ax_0 + by_0 + c|}{\sqrt{a^2 + b^2}}$$

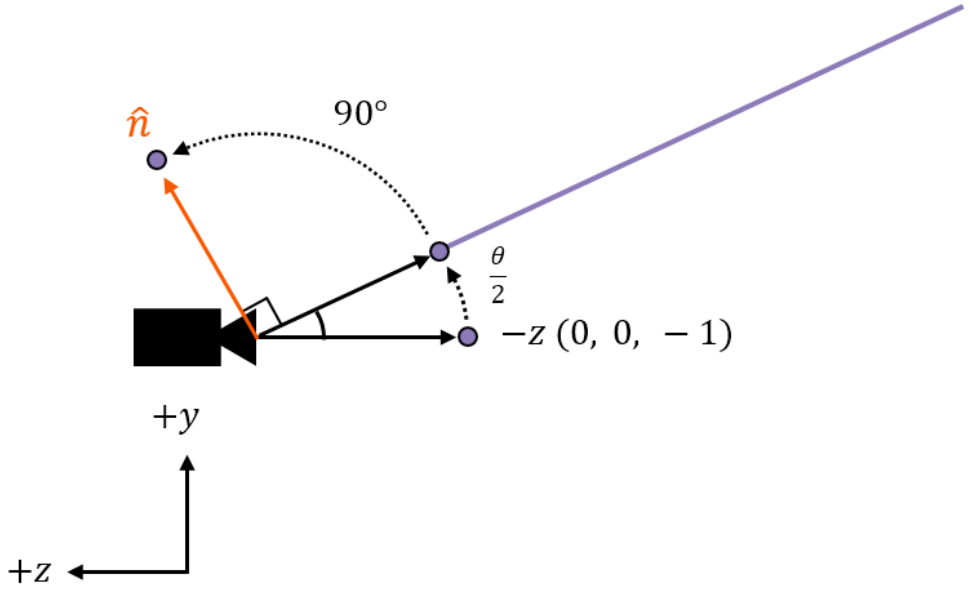


13.1.3 평면의 방정식을 활용한 절두체 표현

- 절두체의 y축 상단 평면



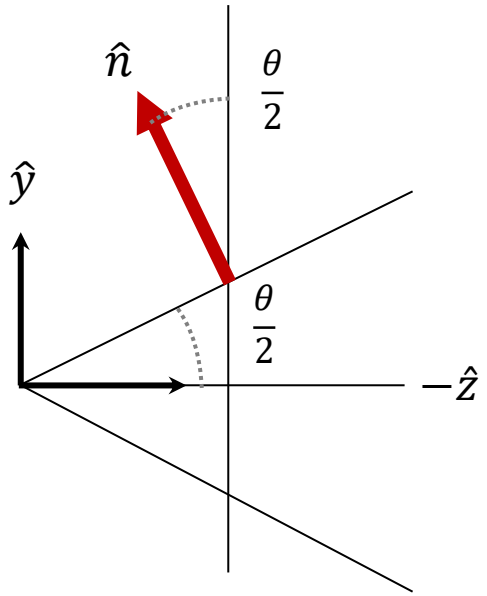
- 절두체 상단 법선 벡터



13.1.3 절두체 평면 구하기

- 절두체의 y축 상단 평면

$$\left(0,\ \sin\frac{\theta}{2},\ -\cos\frac{\theta}{2}\right) \Rightarrow \hat{n} = \left(0,\ \cos\frac{\theta}{2},\ \sin\frac{\theta}{2}\right)$$



- 절두체 y축 상단 평면 방정식

$$\cos\frac{\theta}{2}y + \sin\frac{\theta}{2}z = 0$$

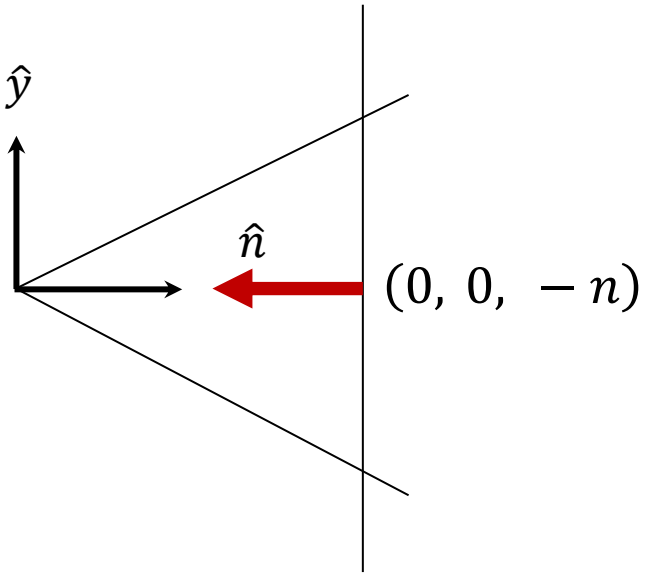
- 절두체 하단 평면 방정식 $\quad -\cos\frac{\theta}{2}y + \sin\frac{\theta}{2}z = 0$
- 절두체 좌측 평면의 방정식 $\quad -\cos\frac{\theta}{2}x + \sin\frac{\theta}{2}z = 0$
- 절두체 우측 평면의 방정식 $\quad \cos\frac{\theta}{2}x + \sin\frac{\theta}{2}z = 0$

13.1.3 절두체 평면 구하기

- **근 평면 방정식**

$$(0,\ 0,\ 1)\cdot(x,\ y,\ z)-(0,\ 0,\ 1)\cdot(0,\ 0,\ -n)=0$$

$$\therefore\ z+n\ =\ 0$$



- **원 평면 방정식**

$$(0,\ 0,\ -1)\cdot(x,\ y,\ z)-(0,\ 0,\ 1)\cdot(0,\ 0,\ f)=0$$

$$\therefore\ -z-\mathrm{f}\ =\ 0$$

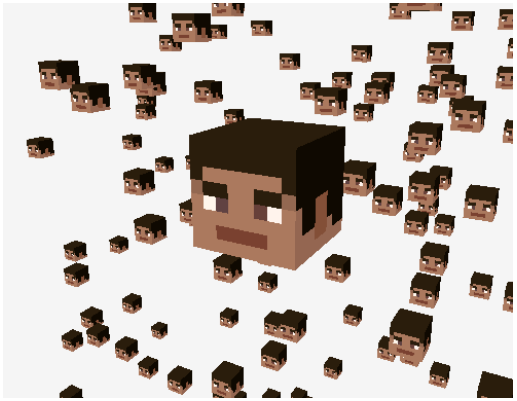
- **주의**
  - **방향 정보가 포함되어 있어서 수식을 함부로 옮기면 안됨**
  - $0=\ z+f\ \rightarrow$ **원평면과 동일하나 평면의 방향이 원평면의 반대**
  - **서로 다른 평면**

$$z+f=0\ \ \neq\ \ -z-f=0$$

실습 예제 13.1 컬링

- 절두체 평면

```
// 절두체를 구성하는 평면의 방정식
static std::array<Plane, 6> frustumPlanes = {
  Plane( Vector3( pCos,   0.f, pSin), 0.f), // +Y
  Plane( Vector3(-pCos,   0.f, pSin), 0.f), // -Y
  Plane( Vector3(  0.f,  pCos, pSin), 0.f), // +X
  Plane( Vector3(  0.f, -pCos, pSin), 0.f), // -X
  Plane( Vector3::UnitZ, nearZ), // +Z
  Plane(-Vector3::UnitZ, -farZ) // -Z
```



- 절두체 안팎 체크

```
BoundCheckResult Frustum::CheckBound(const Vector3& InPoint) const {
  for (const auto& p : Planes) {
    if (p.IsOutside(InPoint)) {
      return BoundCheckResult::Outside;
    }
    else if (Math::EqualsInTolerance(p.Distance(InPoint), 0.f)) {
      return BoundCheckResult::Intersect;
    }
  }
  return BoundCheckResult::Inside;
}
```

```
bool Plane::IsOutside(const Vector3& InPoint) const {
  return Normal.Dot(InPoint) + D > 0.f;
}
```

13.1.4 원근 투영 행렬로부터 평면의 방정식 만들기

- NDC 영역 (Normalized Device Coordinates) 좌표 범위

$$-1 \le n_x \le +1$$
$$-1 \le n_y \le +1$$
$$-1 \le n_z \le +1$$

- w 적용

$$-1 \le \frac{x}{w} \le +1 \qquad -w \le x \le +w$$
$$-1 \le \frac{y}{w} \le +1 \qquad -w \le y \le +w$$
$$-1 \le \frac{z}{w} \le +1 \qquad -w \le z \le +w$$

- **뷰 공간 → 클립 공간 변환**

$$-w \le x \le +w$$
$$-w \le y \le +w$$
$$-w \le z \le +w$$

$$P \cdot \vec{v} = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & \frac{n+f}{n-f} & \frac{2nf}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ v_z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \Rightarrow \begin{bmatrix} P_{row1} \cdot \vec{v} \\ P_{row2} \cdot \vec{v} \\ P_{row3} \cdot \vec{v} \\ P_{row4} \cdot \vec{v} \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}$$

$$\begin{aligned} x &= P_{row1} \cdot \vec{v} \\ y &= P_{row2} \cdot \vec{v} \\ z &= P_{row3} \cdot \vec{v} \\ w &= P_{row4} \cdot \vec{v} \end{aligned} \Rightarrow \begin{aligned} -P_{row4} \cdot \vec{v} \le P_{row1} \cdot \vec{v} \le +P_{row4} \cdot \vec{v} \\ -P_{row4} \cdot \vec{v} \le P_{row2} \cdot \vec{v} \le +P_{row4} \cdot \vec{v} \\ -P_{row4} \cdot \vec{v} \le P_{row3} \cdot \vec{v} \le +P_{row4} \cdot \vec{v} \end{aligned} \Rightarrow \begin{aligned} 0 \le P_{row4} \cdot \vec{v} + P_{row1} \cdot \vec{v} \\ 0 \le P_{row4} \cdot \vec{v} + P_{row1} \cdot \vec{v} \\ 0 \le P_{row4} \cdot \vec{v} + P_{row1} \cdot \vec{v} \end{aligned}$$

$$0 \le (P_{row4} + P_{row1}) \cdot \vec{v} \quad 0 \le (P_{row4} + P_{row2}) \cdot \vec{v} \quad 0 \le (P_{row4} + P_{row3}) \cdot \vec{v}$$
$$0 \le (P_{row4} - P_{row1}) \cdot \vec{v} \quad 0 \le (P_{row4} - P_{row2}) \cdot \vec{v} \quad 0 \le (P_{row4} - P_{row3}) \cdot \vec{v}$$

13.1.4 원근 투영 행렬로부터 평면의 방정식 만들기

- 6개 부등식이 참이면 절두체 영역 내부

$$0 \le (P_{row4} + P_{row1}) \cdot \vec{v} \quad 0 \le (P_{row4} + P_{row2}) \cdot \vec{v} \quad 0 \le (P_{row4} + P_{row3}) \cdot \vec{v}$$
$$0 \le (P_{row4} - P_{row1}) \cdot \vec{v} \quad 0 \le (P_{row4} - P_{row2}) \cdot \vec{v} \quad 0 \le (P_{row4} - P_{row3}) \cdot \vec{v}$$

- 뷰 공간 $\vec{v}$ 절두체 밖 → 부호 반전
  - 평면: a, b, c, d

$$-\left(\frac{ax + by + cz + d}{\sqrt{a^2 + b^2 + c^2}}\right) > 0$$

  - 평면을 정규화 하면 수식이 줄어듦

$$-(ax + by + cz + d) > 0$$

$$\therefore \; ax + by + cz + d \; < 0$$

- **절두체 상단**

$$(P_{row4} - P_{row2}) = (0,\ 0,\ -1,\ 0) - (0,\ d,\ 0,\ 0) = (0,\ -d,\ -1,\ 0)$$

$$(P_{row4} + P_{row2}) \cdot \vec{v} = (0,\ -d,\ -1,\ 0) \cdot (x,\ y,\ z,\ 1)$$

$$= -dy - z$$

$$= -\left(\frac{1}{\tan\frac{\theta}{2}}\right) y - z \quad = -\left(\frac{\cos\frac{\theta}{2}}{\sin\frac{\theta}{2}}\right) y - z$$

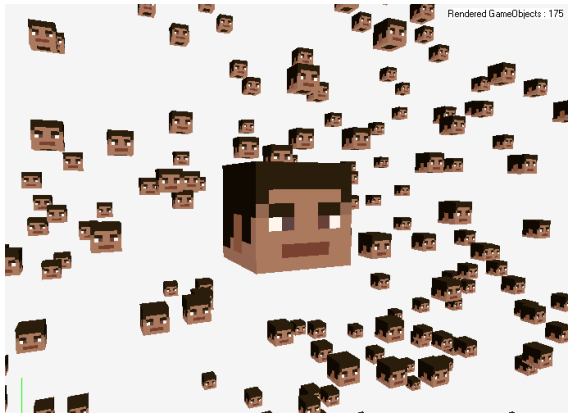
$$= \left(-\cos\frac{\theta}{2} y - \sin\frac{\theta}{2} z\right) \times \frac{1}{\sin\frac{\theta}{2}}$$

$$(P_{row4} + P_{row2}) \cdot \vec{v} = 0$$

$$\left(-\cos\frac{\theta}{2} y - \sin\frac{\theta}{2} z\right) \times \frac{1}{\sin\frac{\theta}{2}} = 0 \quad \Rightarrow \quad \left(-\cos\frac{\theta}{2} y - \sin\frac{\theta}{2} z\right) = 0$$

- 절두체 평면

```
// 절두체를 구성하는 평면의 방정식
std::array<Plane, 6> frustumPlanes = {
Plane(-(ptMatrix[3] - ptMatrix[1])), // +Y
Plane(-(ptMatrix[3] + ptMatrix[1])), // -Y
Plane(-(ptMatrix[3] - ptMatrix[0])), // +X
Plane(-(ptMatrix[3] + ptMatrix[0])), // -X
Plane(-(ptMatrix[3] - ptMatrix[2])), // +Z
Plane(-(ptMatrix[3] + ptMatrix[2])), // -Z
```

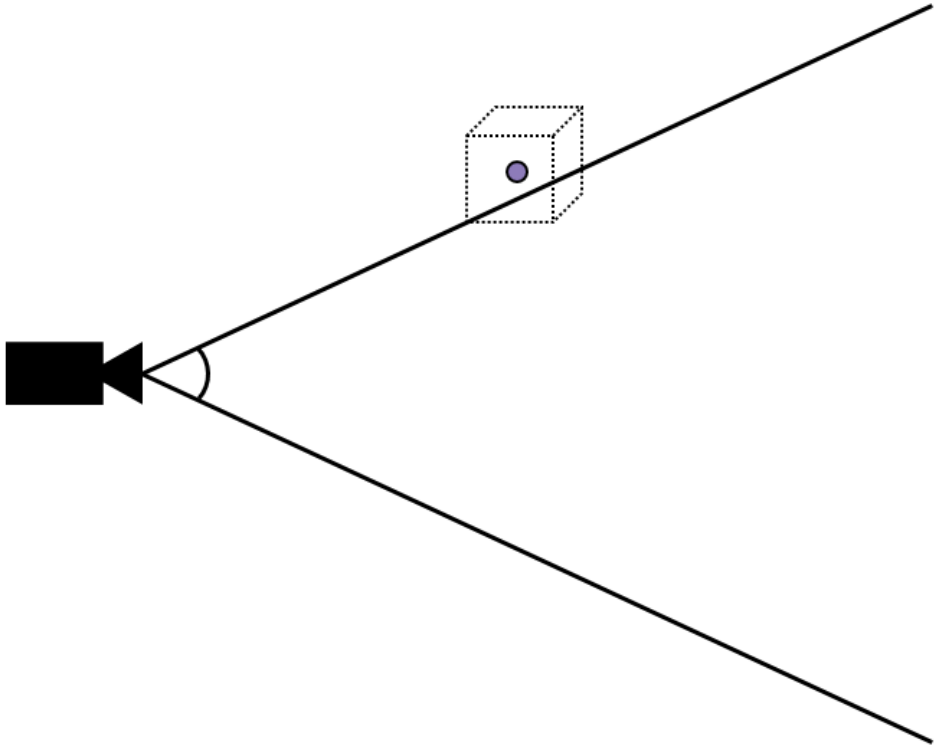


- 절두체 안팎 체크

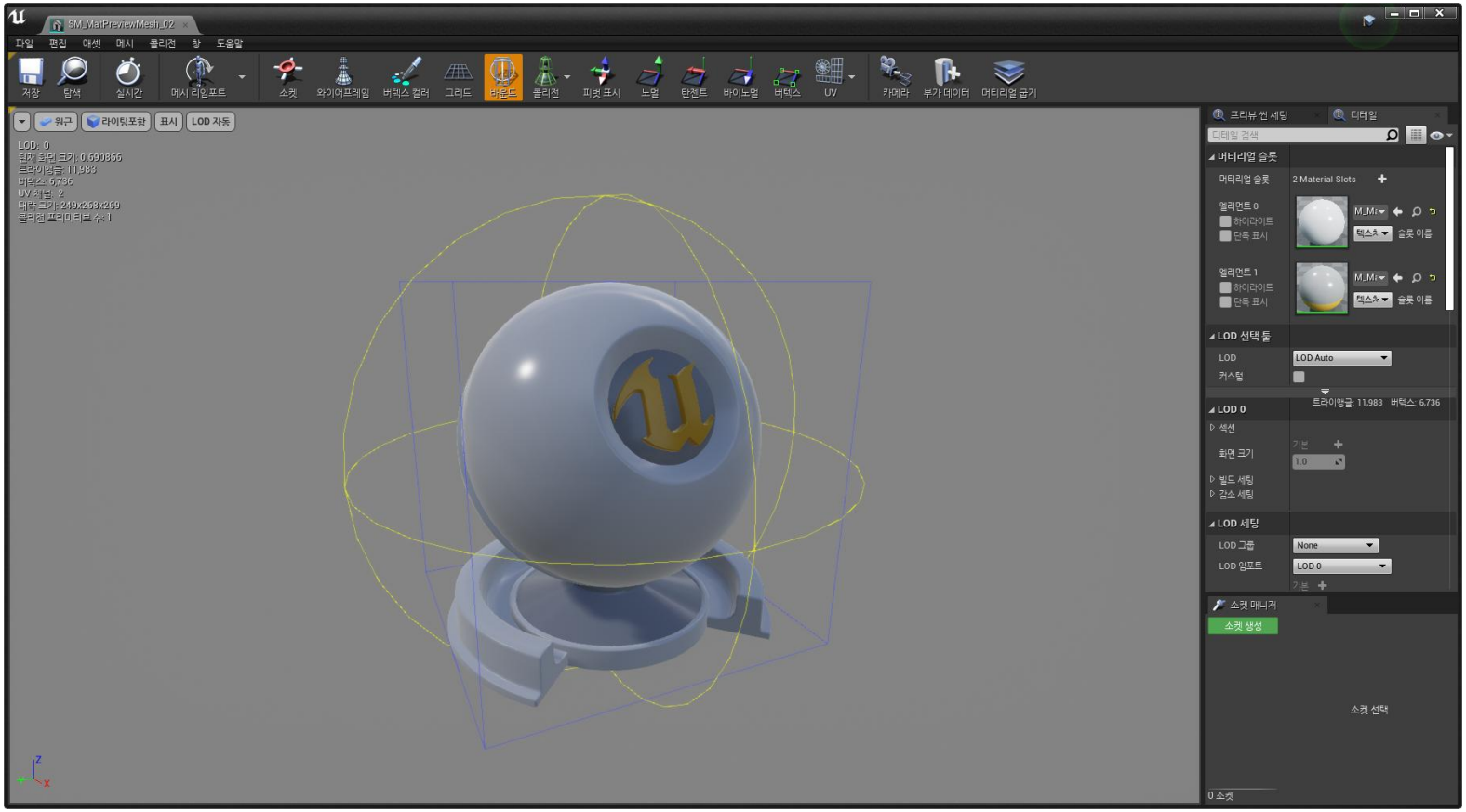
```
BoundCheckResult Frustum::CheckBound(const Vector3& InPoint) const {
  for (const auto& p : Planes) {
    if (p.IsOutside(InPoint)) {
      return BoundCheckResult::Outside;
    }
    else if (Math::EqualsInTolerance(p.Distance(InPoint), 0.f)) {
      return BoundCheckResult::Intersect;
    }
  }
  return BoundCheckResult::Inside;
}
```

- 바운딩 볼륨 (Bounding Volume)
  - 객체를 감싸는 단순 기하 구조 (충돌·가시성 판별용)
  - 충돌·가시성 판별을 위한 오브젝트의 공간 데이터

- 절두체에 걸쳐진 게임 오브젝트



- 언리얼 엔진의 메시 바운딩 볼륨

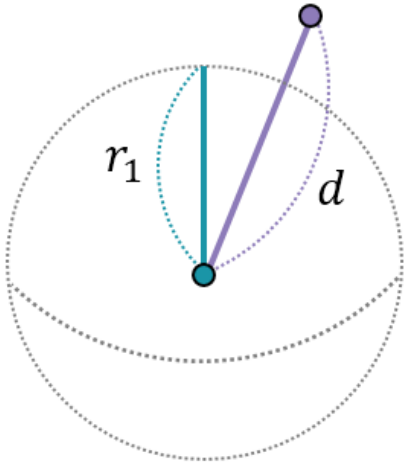


13.2.1 구 바운딩 볼륨의 판정

- 구 바운딩
  - 계산이 빠르고 단순함

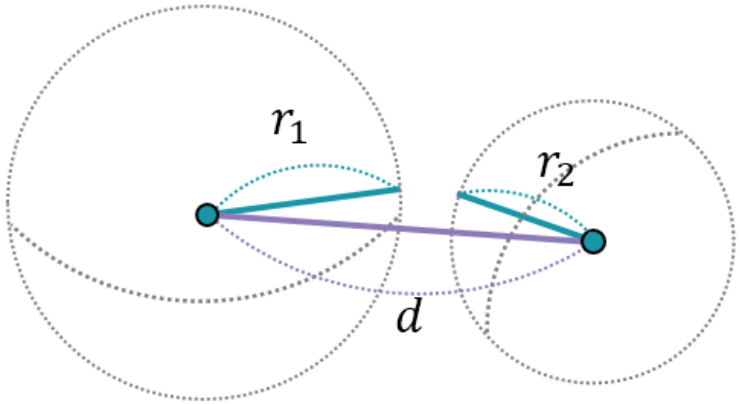
- 구 바운딩 내부

(a)



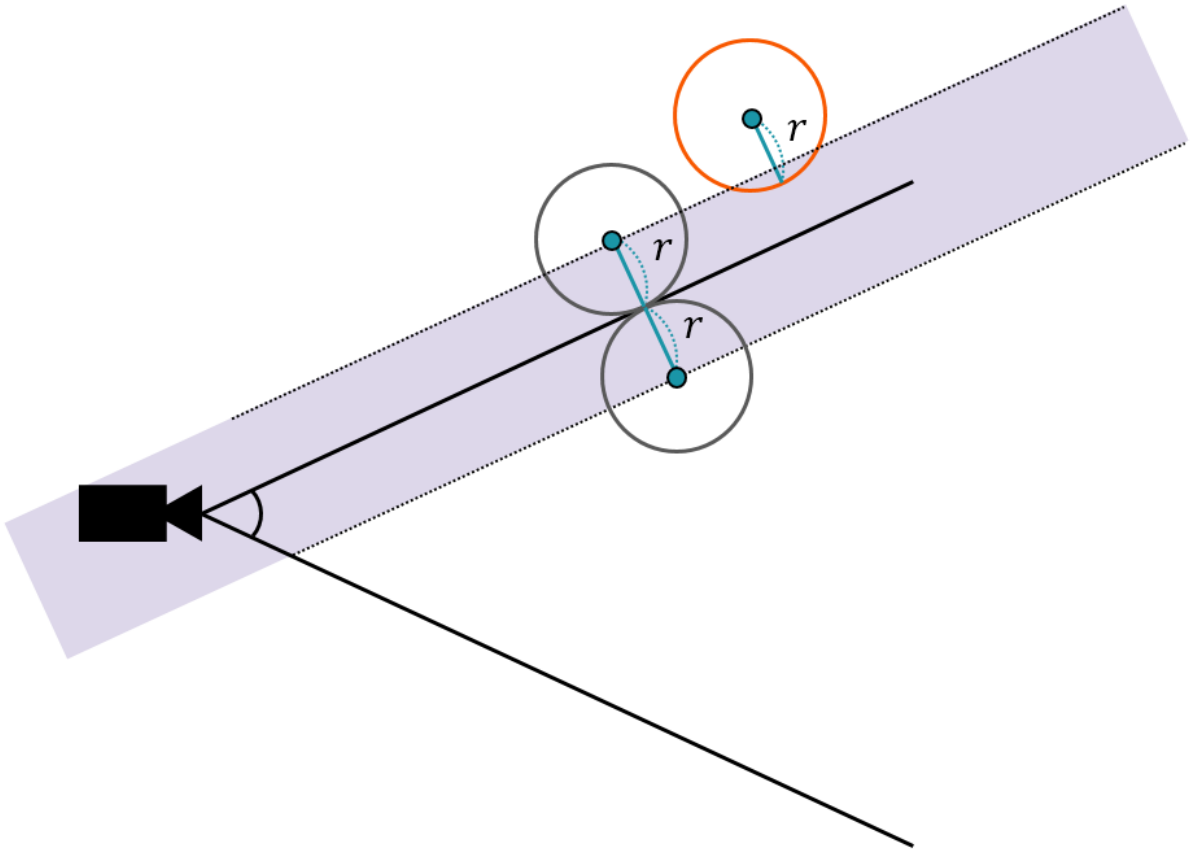
$d < r_1$

(b)



$d < r1 + r2$

- 절두체 영역 vs 구 영역



- Bounding Sphere

```
BoundCheckResult Frustum::CheckBound(const Sphere& InSphere) const
{
  for (const auto& p : Planes)
  {
    if (p.Distance(InSphere.Center) > InSphere.Radius)
    {
      return BoundCheckResult::Outside;
    }

    else if (Math::Abs(p.Distance(InSphere.Center)) <= InSphere.Radius)
    {
      return BoundCheckResult::Intersect;
    }
  }

  return BoundCheckResult::Inside;
}
```

- Clip 좌표

$$\overrightarrow{V_{clip}} = (P) \cdot \overrightarrow{V_{view}}$$

$$\overrightarrow{V_{clip}} = (PVM) \cdot \overrightarrow{V_{local}}$$

```
// 렌더링 로직의 로컬 변수
const Matrix4x4 pvMatrix = mainCamera.GetPerspectiveViewMatrix();
```

```
// 최종 행렬 계산
Matrix4x4 finalMatrix = pvMatrix * transform.GetModelingMatrix();
```

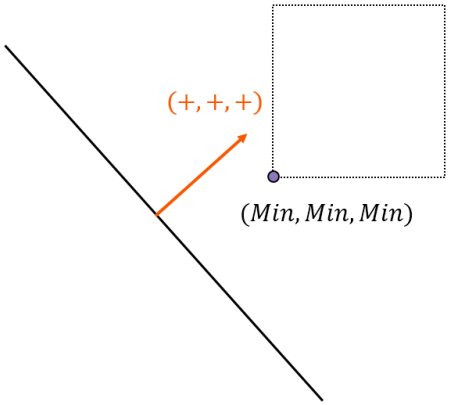
```
std::array<Plane, 6> frustumPlanesFromMatrix = {
Plane(-(finalTransposedMatrix[3] - finalTransposedMatrix[1])), // up
Plane(-(finalTransposedMatrix[3] + finalTransposedMatrix[1])), // bottom
Plane(-(finalTransposedMatrix[3] - finalTransposedMatrix[0])), // right
Plane(-(finalTransposedMatrix[3] + finalTransposedMatrix[0])), // left
Plane(-(finalTransposedMatrix[3] - finalTransposedMatrix[2])),  // far
Plane(-(finalTransposedMatrix[3] + finalTransposedMatrix[2])), // near
};
```

- AABB(Axis Aligned Bounding Box)
  - 축에 정렬된 직육면체 바운딩 볼륨
  - 최소값(min) / 최대값(max) 좌표로 표현
  - 회전 없음 (X, Y, Z 축과 평행)
    → 대신 회전 객체에서는 여유 공간(오차) 발생

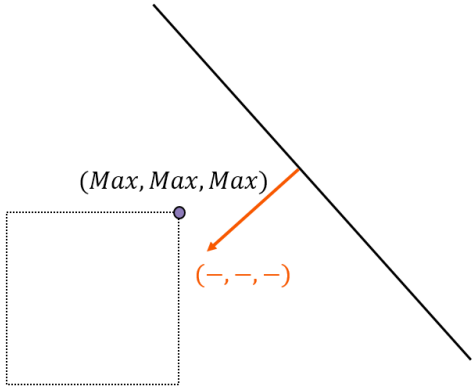
13.2.2 AABB 판정

- 법선 벡터의 부호에 따른 AABB의 비교 판정

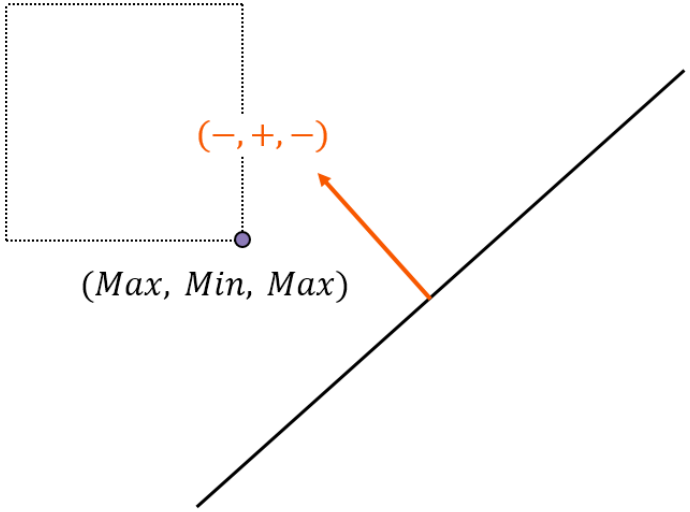
(a)



(b)

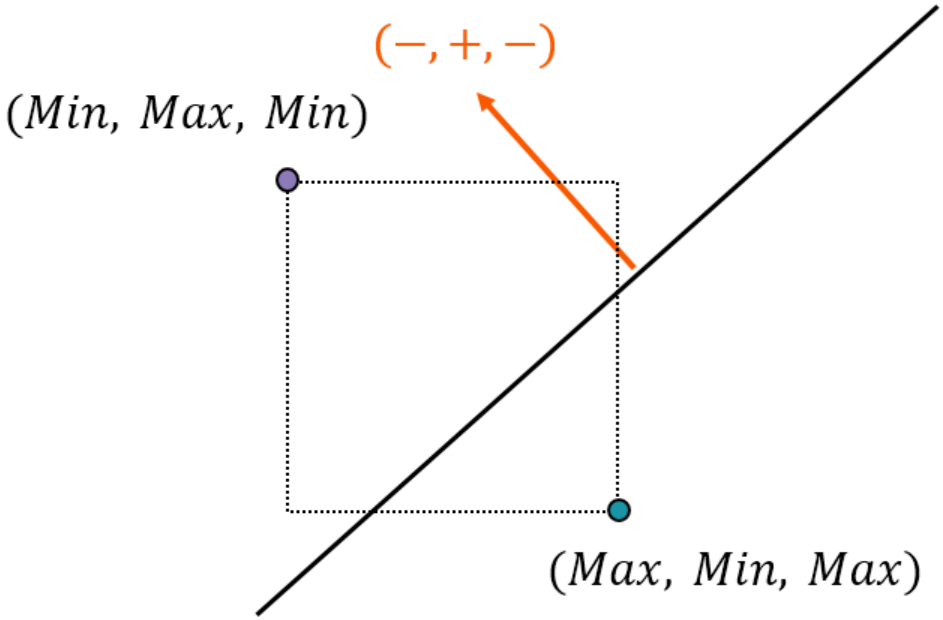


- 한 쪽만 양수인 경우의 판정 방법



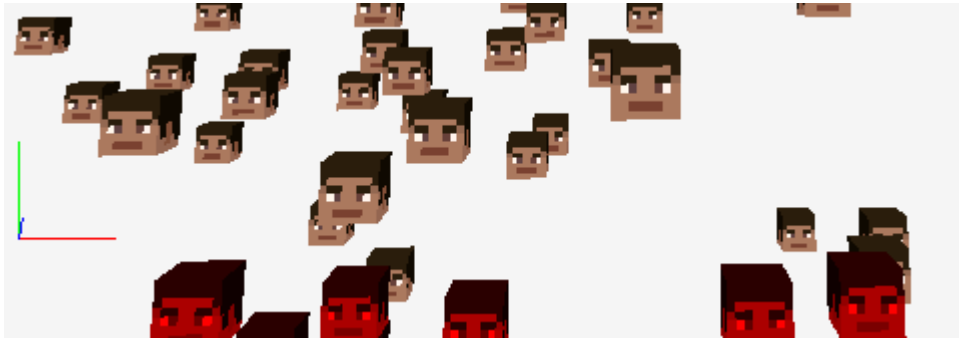
13.2.2 AABB 판정

- AABB영역과 평면이 교차



실습 예제 13-5 박스 영역과 절두체 영역의 판정

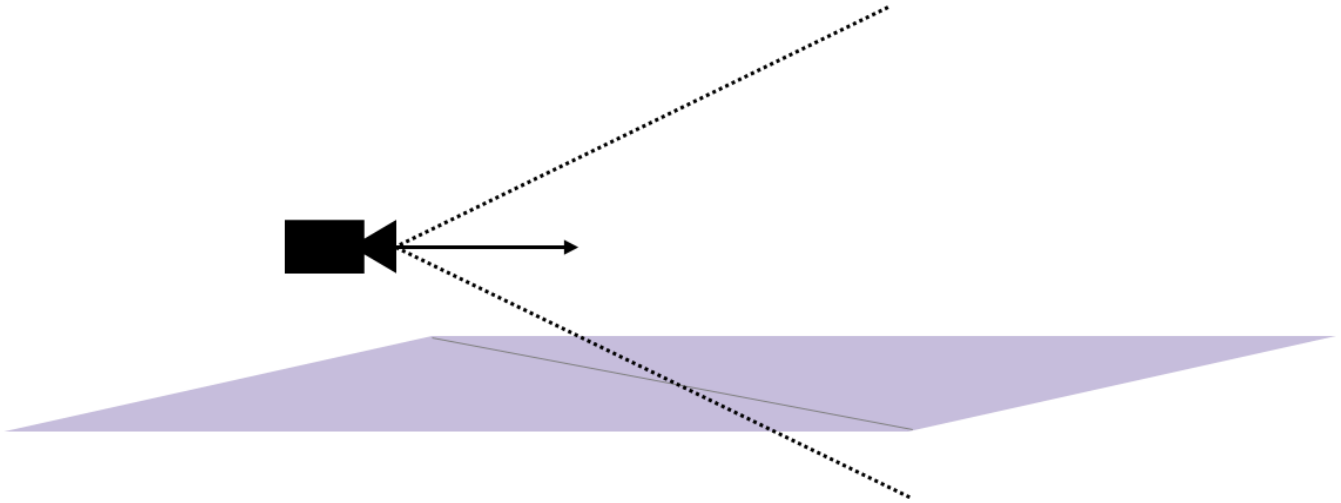
- AABB 판정



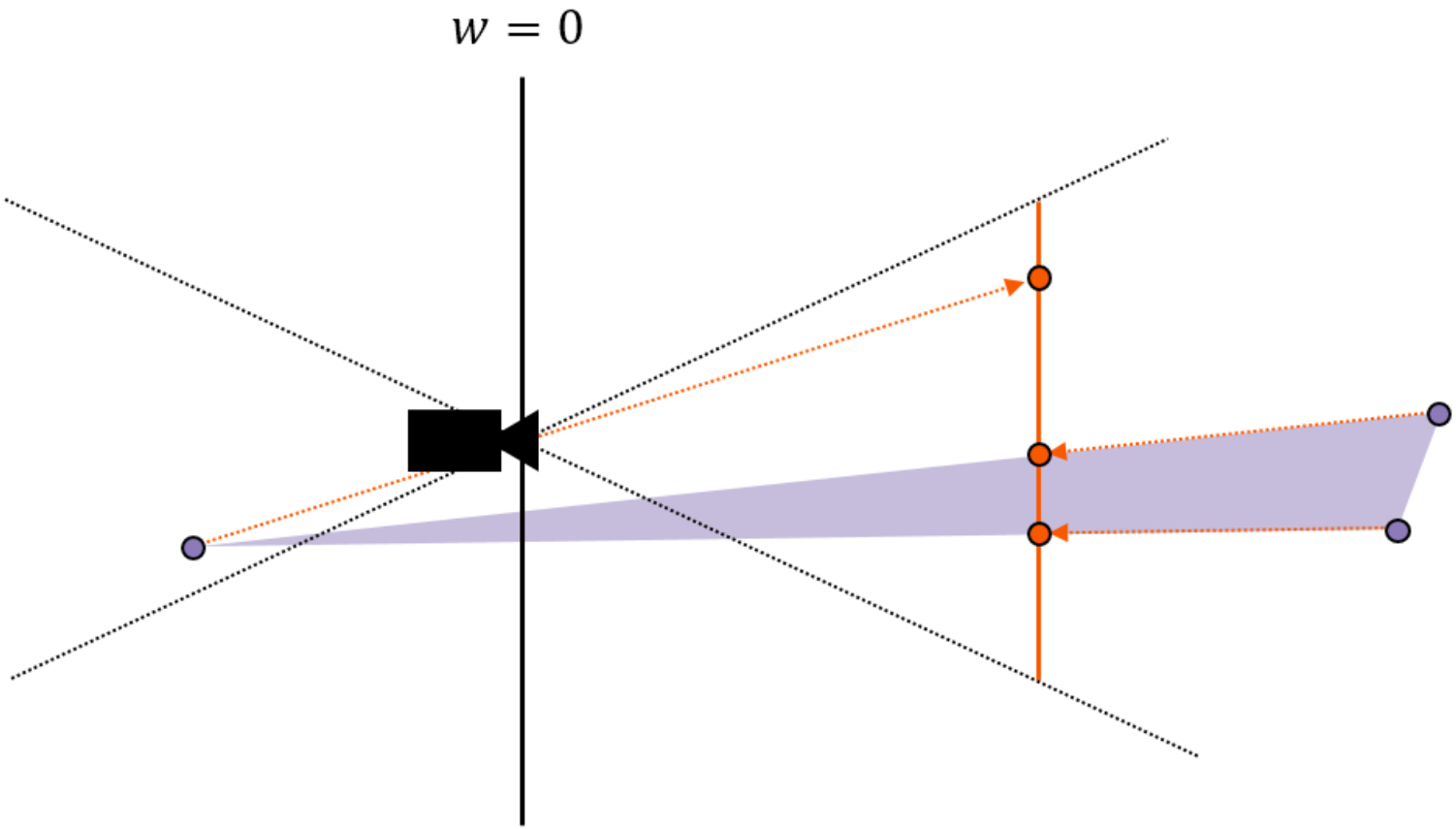
```
BoundCheckResult Frustum::CheckBound(const Box& InBox) const {
  for (const auto& p : Planes) {
    Vector3 pPoint = InBox.Min, nPoint = InBox.Max;
    if (p.Normal.X >= 0.f) { pPoint.X = InBox.Max.X; nPoint.X = InBox.Min.X; }
    if (p.Normal.Y >= 0.f) { pPoint.Y = InBox.Max.Y; nPoint.Y = InBox.Min.Y; }
    if (p.Normal.Z >= 0.f) { pPoint.Z = InBox.Max.Z; nPoint.Z = InBox.Min.Z; }
    if (p.Distance(nPoint) > 0.f)
    {
      return BoundCheckResult::Outside;
    }
    if (p.Distance(nPoint) <= 0.f && p.Distance(pPoint) >= 0.f)
    {
      return BoundCheckResult::Intersect;
    }
  }
  return BoundCheckResult::Inside;
}
```

- 클리핑 문제

- 카메라 바로 아래에 거대한 사각형 메시가 위치한 경우



- 카메라 뒤에 위치한 점을 거꾸로 투영하는 경우의 문제



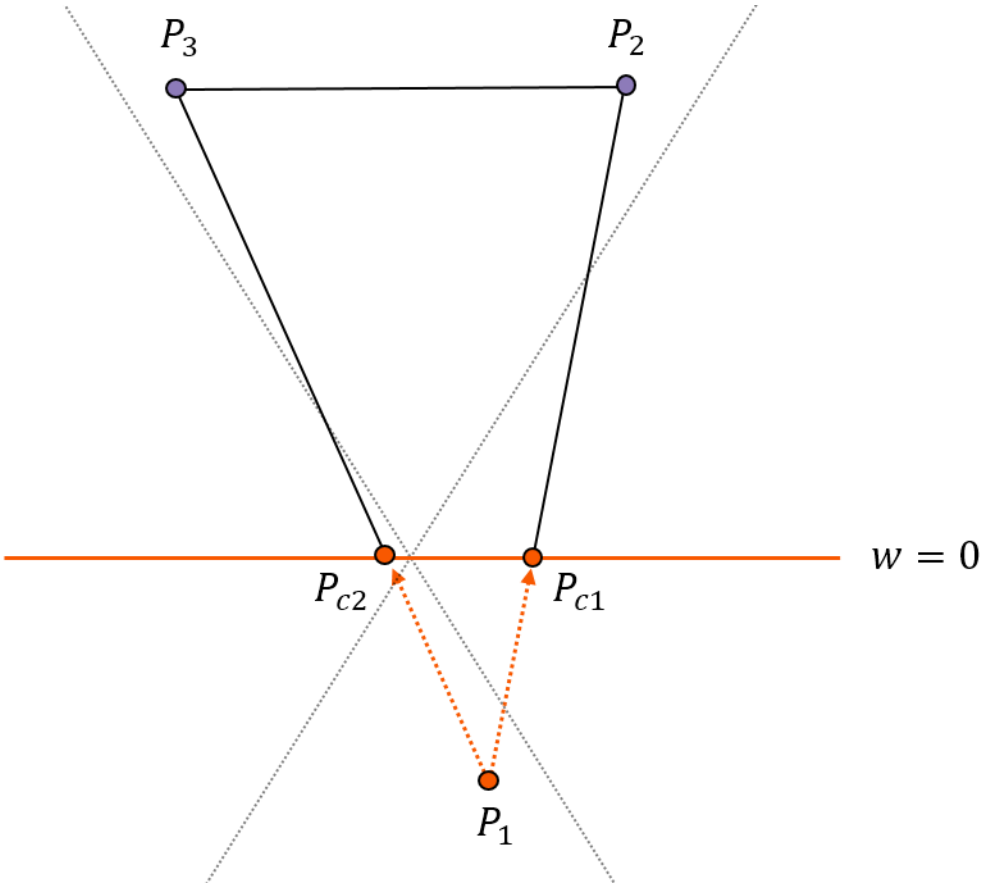
- 문제를 방지하기 위한 사전 작업

$$P_{c1} = P_{1c} \cdot (1 - t_1) + P_2 \cdot t_1$$

$$P_{c1}.w = 0$$

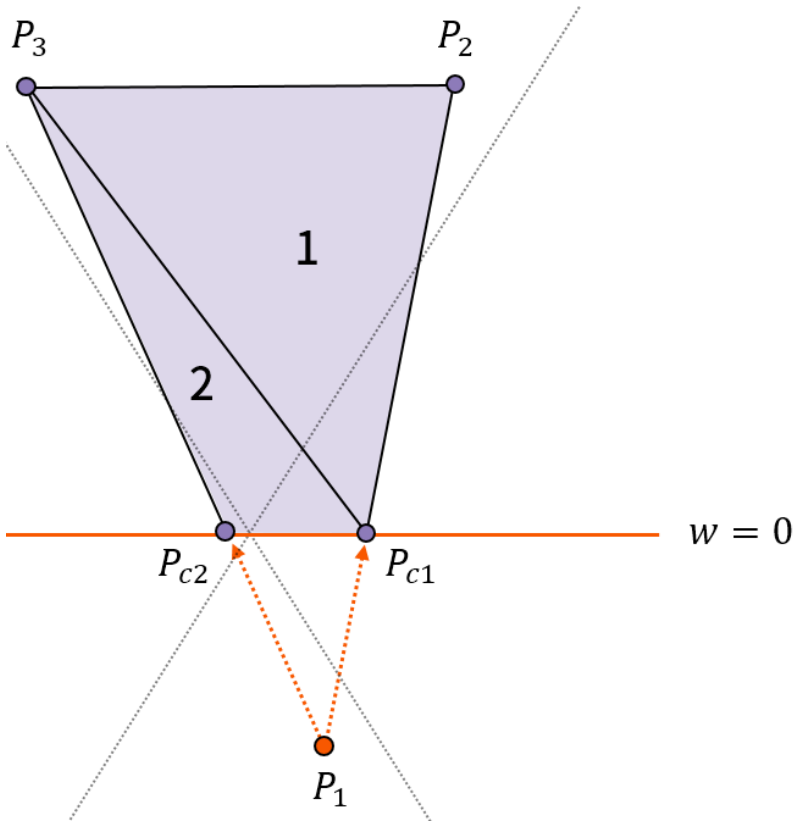
$$w_1(1 - t_1) + w_2 t_1 = 0$$

$$t_1 = \frac{w_1}{w_1 - w_2}$$

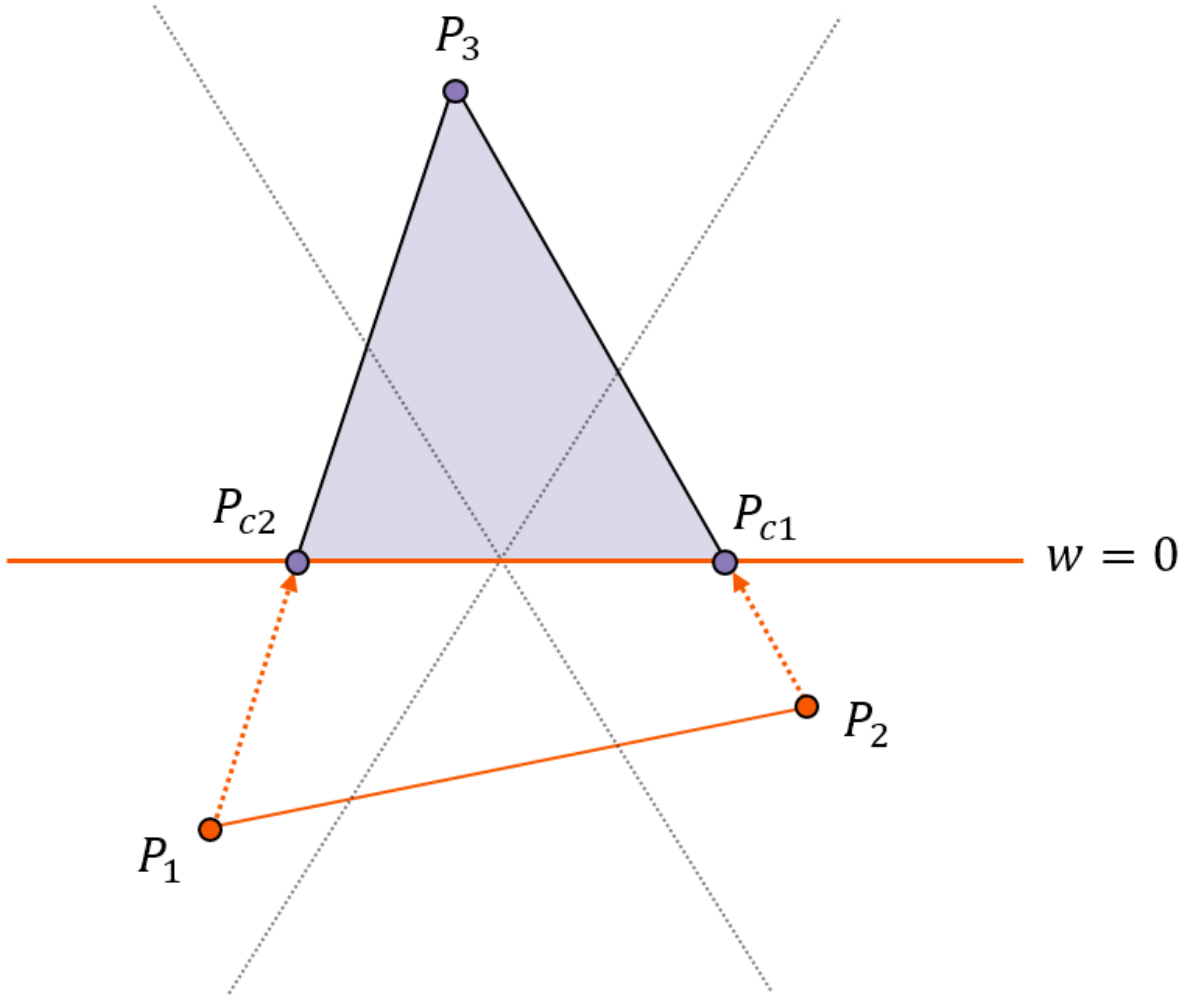


- 사각형을 두 개의 삼각형으로 분할하기

- 삼각형 분할 예
  - 정점 배치 순서
    $P_1 \rightarrow P_2 \rightarrow P_3$
  - 삼각형 1의 순서
    $P_{c1} \rightarrow P_2 \rightarrow P_3$
  - 삼각형 2의 순서
    $P_{c1} \rightarrow P_3 \rightarrow P_{c2}$



- 두 점이 카메라 뒤에 위치한 경우의 분할



- 동차좌표 상에서 삼각형 분할 작업
  - 잘라낼 영역 안의 점의 개수 파악

- 잘라낼 영역의 점의 수
  - 점 0 → 삼각형 출력
  - 점 1 → 평면과의 교차점을 구하고 교차점으로부터 삼각형 두 개로 분할
  - 점 2 → 이 속해 있으면 교차점을 구해 갱신
  - 점 3 → 그리지 않음

- 오른쪽 평면에 대해 추가로 분할한 삼각형의 예시

$$\frac{x}{w} > 1 \quad \therefore x > w$$

$$P_{1.x} = x_1 \quad P_{2.x} = x_2$$

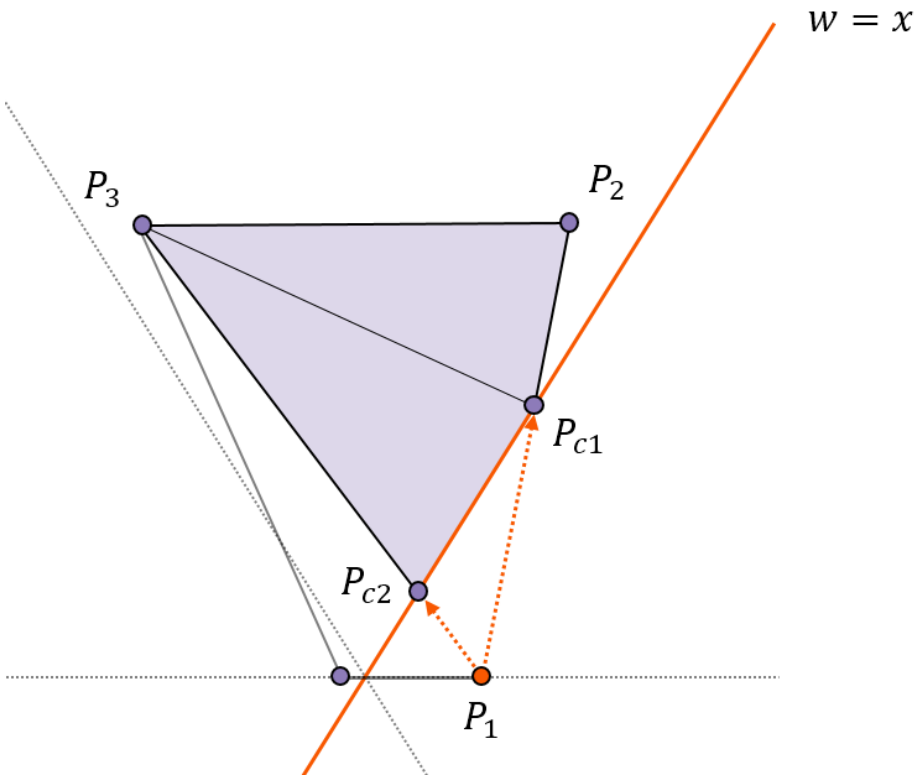
$$x_{c1} = x_1(1 - t_1) + x_2 t_1$$

$$w_{c1} = w_1(1 - t_1) + w_2 t_1$$

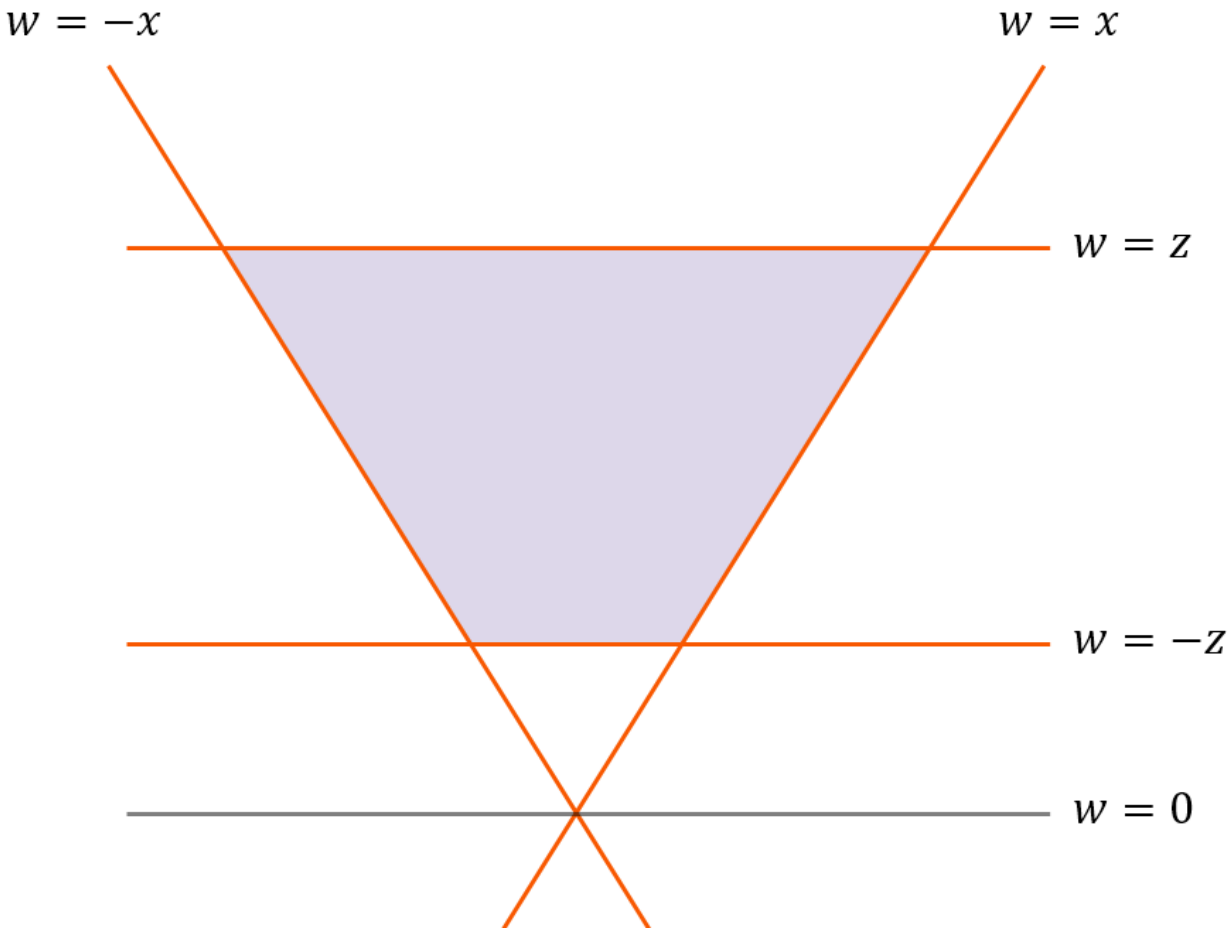
$$x_{c1} = w_{c1}$$

$$x_1(1 - t_1) + x_2 t_1 = w_1(1 - t_1) + w_2 t_1$$

$$t_1 = \frac{(w_1 - x_1)}{(w_1 - x_1) - (w_2 - x_2)}$$



- 절두체에 맞게 삼각형을 잘라내는데 사용할 각 평면의 방정식



- 삼각형 분할을 위한 각 평면의 판별식과 계수

순서	평면	외부 판별식	아핀 결합의 계수
1	$w = 0$	$w < 0$	$\frac{w_1}{w_1 - w_2}$
2	$w = y$	$y > w$	$\frac{(w_1 - y_1)}{(w_1 - y_1) - (w_2 - y_2)}$
3	$w = -y$	$y < -w$	$\frac{(w_1 + y_1)}{(w_1 + y_1) - (w_2 + y_2)}$
4	$w = x$	$x > w$	$\frac{(w_1 - x_1)}{(w_1 - x_1) - (w_2 - x_2)}$
5	$w = -x$	$x < -w$	$\frac{(w_1 + x_1)}{(w_1 + x_1) - (w_2 + x_2)}$
6	$w = z$	$z > w$	$\frac{(w_1 - z_1)}{(w_1 - z_1) - (w_2 - z_2)}$
7	$w = -z$	$z < -w$	$\frac{(w_1 + z_1)}{(w_1 + z_1) - (w_2 + z_2)}$

- 평면의 방정식
- 절두체 생성
- 절두체 컬링
- AABB 컬링
- 클리핑